

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG HỆ THỐNG THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
CÁC HỆ THỐNG PHÂN TÁN
CHỦ ĐỀ:
PHÁT TRIỂN HỆ THỐNG SOẠN THẢO VĂN BẢN CỘNG
TÁC

Giảng viên hướng dẫn	Kim Ngọc Bách
Lớp	D22HTTT3
Nhóm	6
Thành viên	MSV
Phạm Văn Hiếu	B22DCCN317
Ngô Thành Trung	B22DCCN869
Nguyễn Tiến Đạt	B22DCCN197

Hà Nội – 2026

LỜI CẢM ƠN

Nhóm em xin chân thành gửi lời cảm ơn đến thầy Kim Ngọc Bách – giảng viên trực tiếp giảng dạy và hướng dẫn môn học Các hệ thống phân tán. Trong suốt quá trình lên lớp, thầy không chỉ truyền đạt những kiến thức chuyên môn cốt lõi về kiến trúc phân tán mà còn chia sẻ những bài học kinh nghiệm thực tiễn, những câu chuyện quý báu đầy bổ ích. Chính những định hướng đó đã giúp nhóm có được cái nhìn tổng quan về các xu hướng công nghệ trên thế giới, từ đó có nền tảng vững chắc để xây dựng và hoàn thiện bài tập lớn này.

Đề tài "Phát triển hệ thống soạn thảo văn bản cộng tác" là một bài toán thú vị nhưng cũng đặt ra nhiều thách thức về mặt kỹ thuật. Mặc dù đã cố gắng vận dụng tối đa kiến thức đã học và nỗ lực hết sức trong quá trình nghiên cứu, thiết kế và lập trình, nhưng do những hạn chế về mặt thời gian và kinh nghiệm thực tế, báo cáo và sản phẩm của nhóm chắc chắn không thể tránh khỏi những sai sót.

Nhóm vô cùng mong muốn được lắng nghe những lời nhận xét, đánh giá và đóng góp chỉnh sửa quý báu từ thầy để nhóm có thể khắc phục những điểm yếu, hoàn thiện sản phẩm và trau dồi thêm kiến thức cho công việc sau này.

Một lần nữa, nhóm em xin trân trọng cảm ơn!

Nhóm 6

MỤC LỤC

LỜI CẢM ƠN.....	2
MỤC LỤC.....	2
DANH MỤC HÌNH ẢNH.....	5
DANH MỤC BẢNG BIỂU.....	6
CHƯƠNG 1. TỔNG QUAN VÀ PHÂN TÍCH YÊU CẦU.....	8
1.1. Lý do chọn đề tài.....	8
1.2. Mục tiêu của đề tài.....	8
1.2.1. Mục tiêu chức năng.....	8
1.2.2. Mục tiêu kỹ thuật.....	9
1.3. Phạm vi thực hiện.....	9
1.3.1. Phạm vi chức năng được triển khai.....	9
1.3.2. Phạm vi kỹ thuật.....	10
1.4. Công nghệ sử dụng.....	10
1.4.1. Frontend: React, TipTap.....	10
1.4.2. Realtime: Socket.IO, Yjs.....	11
1.4.3. Backend: Node.js, Express.....	12
1.4.4. Database: MySQL, Sequelize ORM.....	12
1.5. Phân tích yêu cầu chức năng.....	13
1.5.1. Quản lý người dùng và đăng nhập.....	13
1.5.2. Quản lý tài liệu.....	14
1.5.3. Soạn thảo rich text.....	15
1.5.4. Cộng tác thời gian thực.....	15
1.5.5. Phân quyền tài liệu.....	16
1.5.6. Lịch sử phiên bản.....	17
1.5.7. Tạo bản sao tài liệu.....	17
1.6. Phân tích yêu cầu phi chức năng.....	18
1.6.1. Bảo mật.....	18
1.6.2. Tính nhất quán dữ liệu.....	19
1.6.3. Khả năng mở rộng.....	19
1.6.4. Khả năng phục hồi khi mất kết nối.....	20
CHƯƠNG 2. THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG.....	21
2.1. Kiến trúc tổng thể hệ thống.....	21
2.1.1. Lớp giao diện người dùng (Frontend / Client).....	21
2.1.2. Lớp xử lý nghiệp vụ (Backend / Server).....	21
2.1.3. Lớp lưu trữ dữ liệu (Database / Storage).....	22
2.1.4. Luồng xử lý và cơ chế đồng bộ (Synchronization).....	22
2.2. Thiết kế backend.....	22

2.2.1. Kiến trúc module backend.....	22
2.2.2. Luồng xử lý API.....	23
2.2.3. Xác thực người dùng bằng JWT.....	23
2.2.4. Phân quyền owner, editor, viewer.....	23
2.2.5. Thiết kế REST API.....	24
2.2.6. Thiết kế Socket.IO backend.....	25
2.3. Thiết kế frontend.....	26
2.3.1. Cấu trúc giao diện React.....	27
2.3.2. Trang đăng nhập và đăng ký.....	28
2.3.3. Trang danh sách tài liệu.....	29
2.3.4. Trang soạn thảo tài liệu.....	30
2.3.5. Toolbar rich text.....	30
2.3.6. Phần lịch sử phiên bản.....	31
2.3.7. Phần thiết lập quyền cộng tác.....	32
2.4. Thiết kế cơ sở dữ liệu.....	32
2.4.1. Bảng users.....	33
2.4.2. Bảng documents.....	33
2.4.3. Bảng document_collaborators.....	34
2.4.4. Bảng document_versions.....	35
2.4.5. Bảng document_updates.....	35
2.4.6. Bảng refresh_tokens và blacklisted_tokens.....	36
2.5. Thiết kế lưu trữ nội dung tài liệu.....	36
2.5.1. Lưu content_text.....	37
2.5.2. Lưu content_json.....	37
2.5.3. Lưu content_html.....	37
2.5.4. Lưu ydoc_state.....	38
2.6. Thiết kế cơ chế đồng bộ realtime.....	38
2.6.1. Cơ chế Yjs update.....	38
2.6.2. Cơ chế join document room.....	39
2.6.3. Cơ chế user online và cursor.....	40
2.6.4. Cơ chế offline sync.....	41
2.6.5. Cơ chế undo/redo trong cộng tác.....	42
CHƯƠNG 3. KẾT QUẢ CÀI ĐẶT VÀ ĐÁNH GIÁ.....	43
3.1. Môi trường cài đặt.....	43
3.2. Cấu hình và chạy hệ thống.....	44
3.2.1. Cài đặt với Docker.....	44
3.2.2. Cài đặt trên Local (Không dùng Docker).....	44
3.3. Kết quả đạt được.....	44

3.3.1. Về mặt chức năng nghiệp vụ.....	44
3.3.2. Về mặt kỹ thuật và kiến trúc phân tán.....	45
3.4. Hạn chế của hệ thống.....	46
3.5. Hướng phát triển.....	47
3.6. Kết luận chung.....	49

DANH MỤC HÌNH ẢNH

Hình 2.1. Giao diện đăng nhập/đăng ký.....	28
Hình 2.2. Giao diện trang danh sách tài liệu.....	29
Hình 2.3. Giao diện trang soạn thảo tài liệu.....	30
Hình 2.4. Giao diện toolbar.....	30
Hình 2.5. Giao diện phần lịch sử chỉnh sửa phiên bản.....	31
Hình 2.6. Giao diện phần thiết lập quyền cộng tác.....	32
Hình 2.7. Bảng cơ sở dữ liệu quan hệ.....	33

DANH MỤC BẢNG BIỂU

Bảng 2.1. Bảng phân quyền người dùng.....	24
Bảng 2.2. Bảng users.....	33
Bảng 2.3. Bảng documents.....	34
Bảng 2.4. Bảng document_collaborators.....	34
Bảng 2.5. Bảng document_versions.....	35
Bảng 2.6. Bảng document_updates.....	35
Bảng 2.7. Bảng refresh_tokens.....	36
Bảng 2.8. Bảng blacklisted_tokens.....	36

CHƯƠNG 1. TỔNG QUAN VÀ PHÂN TÍCH YÊU CẦU

1.1. Lý do chọn đề tài

Trong bối cảnh chuyển đổi số ngày càng mạnh mẽ, nhu cầu làm việc nhóm từ xa và cộng tác trực tuyến đã trở thành một phần không thể thiếu trong hoạt động của các tổ chức, doanh nghiệp và cơ sở giáo dục. Các công cụ soạn thảo tài liệu truyền thống như Microsoft Word hay LibreOffice Writer vốn được thiết kế cho môi trường đơn người dùng, do đó thiếu khả năng hỗ trợ nhiều người cùng chỉnh sửa một tài liệu theo thời gian thực. Điều này gây ra không ít bất tiện khi các nhóm làm việc phải chia sẻ file qua email, quản lý nhiều phiên bản thủ công và dễ dẫn đến xung đột dữ liệu.

Sự thành công của Google Docs đã chứng minh tính khả thi và nhu cầu rõ ràng của hệ thống soạn thảo văn bản cộng tác thời gian thực. Tuy nhiên, việc xây dựng một hệ thống tương tự từ đầu vẫn là một thách thức kỹ thuật đáng kể, đặc biệt liên quan đến việc đồng bộ hóa nội dung khi nhiều người dùng chỉnh sửa đồng thời mà không gây ra xung đột hay mất dữ liệu.

Xuất phát từ những lý do trên, đề tài "Xây dựng hệ thống soạn thảo văn bản cộng tác thời gian thực" được lựa chọn nhằm nghiên cứu, thiết kế và triển khai một hệ thống hoàn chỉnh có khả năng hỗ trợ nhiều người dùng cùng làm việc trên một tài liệu, đảm bảo tính nhất quán dữ liệu, hỗ trợ văn bản định dạng giàu, quản lý phiên bản và phân quyền người dùng.

1.2. Mục tiêu của đề tài

Đề tài hướng đến việc xây dựng một hệ thống soạn thảo văn bản cộng tác có đầy đủ các chức năng cốt lõi, cụ thể bao gồm

1.2.1. Mục tiêu chức năng

- Xây dựng hệ thống cho phép nhiều người dùng đăng nhập, tạo, mở, chỉnh sửa và xóa tài liệu.
- Hỗ trợ soạn thảo văn bản định dạng giàu bao gồm in đậm, in nghiêng, gạch chân, tiêu đề các cấp, căn lề, danh sách có thứ tự và không có thứ tự, chèn liên kết, thay đổi màu chữ và highlight văn bản.

- Đồng bộ thay đổi theo thời gian thực giữa nhiều client đang mở cùng một tài liệu, đảm bảo mỗi thao tác của một người dùng được phản ánh gần như ngay lập tức trên màn hình của những người dùng còn lại.
- Hiển thị danh sách người dùng đang trực tuyến, vị trí con trỏ và vùng chọn văn bản của từng người dùng trong tài liệu.
- Hỗ trợ phân quyền ba cấp độ: chủ sở hữu (owner), biên tập viên (editor) và người xem (viewer).
- Lưu lịch sử phiên bản tài liệu và cho phép xem lại hoặc khôi phục phiên bản cũ.
- Hỗ trợ chỉnh sửa ngoại tuyến (offline editing) và tự động đồng bộ khi kết nối được khôi phục.

1.2.2. Mục tiêu kỹ thuật

- Nghiên cứu và ứng dụng cơ chế CRDT thông qua thư viện Yjs để giải quyết bài toán đồng bộ hóa phân tán mà không cần máy chủ trung gian xử lý xung đột.
- Thiết kế kiến trúc hệ thống rõ ràng, tách biệt giữa frontend, backend và cơ sở dữ liệu, đảm bảo khả năng mở rộng.
- Lưu trữ nội dung tài liệu dưới nhiều dạng biểu diễn (JSON, HTML, plain text, Yjs state) để phục vụ các mục đích khác nhau như hiển thị, tìm kiếm, export và phục hồi cộng tác.

1.3. Phạm vi thực hiện

Để đảm bảo tính khả thi trong khuôn khổ thời gian thực hiện đề tài, hệ thống được giới hạn trong các phạm vi sau:

1.3.1. Phạm vi chức năng được triển khai

- Xác thực người dùng cơ bản (đăng ký, đăng nhập) bằng email và mật khẩu; không triển khai đăng nhập qua mạng xã hội hay xác thực hai yếu tố.
- Soạn thảo rich text với bộ tính năng định dạng cơ bản đến trung cấp như đã liệt kê ở mục tiêu; không triển khai các tính năng nâng cao như nhúng bảng tính, vẽ sơ đồ hay nhúng media.

- Cộng tác thời gian thực trên nền tảng web; không có ứng dụng di động native.
- Quản lý phiên bản theo dạng snapshot thủ công; không triển khai diff trực quan giữa các phiên bản.
- Phân quyền ở cấp tài liệu; không triển khai phân quyền theo từng đoạn văn hay từng phần của tài liệu.

1.3.2. Phạm vi kỹ thuật

- Hệ thống được triển khai và kiểm thử trên môi trường localhost và mạng nội bộ; không yêu cầu triển khai production với tải lớn.
- Cơ sở dữ liệu sử dụng MySQL đơn node; không triển khai replication hay clustering.
- Không bao gồm tính năng tìm kiếm toàn văn, xuất file Word/PDF hay tích hợp với dịch vụ lưu trữ đám mây bên thứ ba.

1.4. Công nghệ sử dụng

Hệ thống được xây dựng theo kiến trúc client-server, trong đó frontend và backend giao tiếp qua cả HTTP REST API lẫn kết nối WebSocket theo thời gian thực. Các công nghệ được lựa chọn dựa trên tiêu chí phổ biến trong cộng đồng, hỗ trợ tốt cho bài toán cộng tác thời gian thực và có tài liệu đầy đủ.

1.4.1. Frontend: React, TipTap

React là thư viện JavaScript do Meta phát triển, được sử dụng rộng rãi để xây dựng giao diện người dùng theo mô hình component. React quản lý trạng thái giao diện thông qua cơ chế Virtual DOM, giúp cập nhật giao diện hiệu quả khi dữ liệu thay đổi. Trong hệ thống này, React đảm nhiệm việc xây dựng các trang danh sách tài liệu, trang soạn thảo, thanh công cụ định dạng (toolbar), danh sách người dùng trực tuyến và các trạng thái kết nối.

TipTap là một framework soạn thảo văn bản mã nguồn mở được xây dựng trên nền ProseMirror, cung cấp giao diện lập trình hướng component thân thiện với React. TipTap hỗ trợ kiến trúc mở rộng dựa trên các Extension, cho phép bật/tắt từng tính năng định dạng một cách linh hoạt. Các Extension được sử dụng trong hệ thống bao gồm StarterKit (cung cấp các tính năng cơ bản như paragraph, heading, bold, italic,

list, history), Collaboration và CollaborationCursor (tích hợp với Yjs để đồng bộ nội dung và hiển thị con trỏ của người dùng khác), Underline, TextAlign, Link, TextStyle, Color và Highlight.

Điểm quan trọng là TipTap biểu diễn nội dung tài liệu dưới dạng cấu trúc JSON (ProseMirror document), cho phép lưu trữ và phục hồi đầy đủ định dạng. Khi cần lưu, TipTap cung cấp các phương thức `editor.getJSON()`, `editor.getHTML()` và `editor.getText()` để lấy nội dung theo từng định dạng phục vụ mục đích khác nhau.

1.4.2. Realtime: Socket.IO, Yjs

Yjs là một thư viện triển khai cơ chế CRDT (Conflict-free Replicated Data Type) cho môi trường JavaScript. CRDT là cấu trúc dữ liệu đặc biệt trong đó mọi thay đổi từ các client độc lập đều có thể được hợp nhất một cách tự động và nhất quán mà không cần có máy chủ trung gian phân xử xung đột. Trong bối cảnh soạn thảo cộng tác, Yjs duy trì một Y.Doc (Yjs document) được chia sẻ giữa tất cả các client. Mỗi khi người dùng thực hiện thay đổi, Yjs tạo ra một gói cập nhật nhỏ (update binary) đại diện cho thay đổi đó. Gói này được truyền đến các client khác và áp dụng lên Y.Doc của họ, đảm bảo tất cả các client đạt đến cùng trạng thái cuối cùng bất kể thứ tự nhận được các update.

Yjs còn cung cấp Y.UndoManager để hỗ trợ undo/redo có phạm vi — nghĩa là thao tác undo của một người dùng chỉ hoàn tác thay đổi của chính họ chứ không ảnh hưởng đến thay đổi của người dùng khác. Ngoài ra, khả năng kết hợp với IndexedDB của Yjs cho phép lưu trạng thái Y.Doc ngay trên trình duyệt, hỗ trợ chỉnh sửa ngoại tuyến và đồng bộ lại khi có kết nối.

Socket.IO là thư viện cung cấp giao tiếp hai chiều, theo sự kiện (event-driven) giữa client và server, xây dựng trên nền WebSocket với cơ chế fallback tự động về HTTP long-polling khi WebSocket không khả dụng. Trong hệ thống, Socket.IO đảm nhận vai trò truyền tải các Yjs update giữa các client, phát tán thông tin về trạng thái con trỏ và vùng chọn (awareness), thông báo sự kiện người dùng vào/rời tài liệu, và xử lý các tình huống mất kết nối hay đồng bộ lại sau khi offline. Socket.IO hỗ trợ khái niệm "room" cho phép server chỉ broadcast sự kiện đến các client đang cùng làm việc trên một tài liệu cụ thể, tránh phát tán thông tin không cần thiết.

1.4.3. Backend: Node.js, Express

Node.js là môi trường chạy JavaScript phía máy chủ, được xây dựng trên V8 engine của Chrome. Node.js sử dụng mô hình I/O bất đồng bộ, không chặn (non-blocking I/O) và vòng lặp sự kiện (event loop), giúp xử lý hiệu quả số lượng lớn kết nối đồng thời — đặc biệt phù hợp với các ứng dụng yêu cầu nhiều kết nối WebSocket dài hạn như hệ thống cộng tác thời gian thực. Việc sử dụng JavaScript cả ở frontend lẫn backend cũng giúp giảm chi phí chuyển đổi ngữ cảnh và cho phép tái sử dụng một số logic giữa hai phía.

Express là framework web tối giản và linh hoạt cho Node.js, cung cấp các tính năng cơ bản để xây dựng HTTP API như routing, middleware, xử lý request/response. Trong hệ thống, Express đảm nhận việc định nghĩa các REST API endpoint cho xác thực người dùng, quản lý tài liệu, phân quyền cộng tác và quản lý phiên bản. Các middleware của Express được dùng để xác thực JWT token, kiểm tra quyền truy cập trước khi xử lý mỗi request, và xử lý lỗi tập trung. Socket.IO server được tích hợp cùng với HTTP server của Express để chia sẻ cùng một cổng kết nối.

1.4.4. Database: MySQL, Sequelize ORM

MySQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở phổ biến, sử dụng ngôn ngữ SQL tiêu chuẩn. MySQL được lựa chọn do tính ổn định, hỗ trợ tốt kiểu dữ liệu JSON native (từ phiên bản 5.7.8 trở đi), hỗ trợ LONGBLOB để lưu trữ dữ liệu nhị phân như Yjs state, và có công cụ quản trị phong phú. Hệ thống sử dụng MySQL để lưu trữ thông tin người dùng, metadata tài liệu, nội dung tài liệu theo nhiều định dạng (content_json, content_html, content_text, ydoc_state), thông tin phân quyền cộng tác, lịch sử phiên bản và các Yjs update nhỏ.

Thiết kế lưu trữ đa định dạng là một điểm đáng chú ý: content_json lưu cấu trúc JSON của TipTap là nguồn chân lý cho định dạng rich text; content_text phục vụ preview và tìm kiếm; content_html phục vụ hiển thị nhanh hoặc export; còn ydoc_state lưu trạng thái Yjs binary để phục hồi chính xác phiên cộng tác khi cần.

Sequelize là thư viện ORM (Object-Relational Mapping) cho Node.js, hỗ trợ MySQL cùng nhiều hệ quản trị cơ sở dữ liệu khác. Sequelize cho phép định nghĩa các model JavaScript tương ứng với bảng cơ sở dữ liệu, thực hiện truy vấn thông qua các

phương thức JavaScript thay vì viết SQL thuần túy, và quản lý quan hệ giữa các bảng (associations) như một-nhiều, nhiều-nhiều. Trong hệ thống, Sequelize được dùng để định nghĩa các model User, Document, DocumentCollaborator, DocumentVersion và DocumentUpdate, đồng thời quản lý migration để duy trì schema cơ sở dữ liệu theo phiên bản. Việc sử dụng ORM giúp code backend dễ đọc, dễ bảo trì và giảm nguy cơ SQL injection so với việc xây dựng câu truy vấn thủ công.

1.5. Phân tích yêu cầu chức năng

1.5.1. Quản lý người dùng và đăng nhập

Chức năng quản lý người dùng và đăng nhập được thiết kế như một nền tảng cốt lõi nhằm xác thực danh tính và đảm bảo tính an toàn cho mọi hoạt động trong hệ thống cộng tác. Quy trình khởi đầu với việc đăng ký tài khoản, nơi người dùng cung cấp các thông tin định danh thiết yếu như tên, địa chỉ email và mật khẩu. Để bảo vệ thông tin cá nhân trước các nguy cơ bảo mật, toàn bộ mật khẩu người dùng đều được hệ thống xử lý qua hàm băm (hashing) an toàn trước khi lưu trữ chính thức vào bảng dữ liệu users trong cơ sở dữ liệu MySQL.

Khi người dùng thực hiện thao tác đăng nhập, hệ thống sẽ thực hiện đối chiếu thông tin đầu vào với cơ sở dữ liệu để xác thực tính hợp lệ. Nếu quá trình kiểm tra thành công, backend sẽ cấp phát đồng thời một cặp mã bảo mật gồm Access Token và Refresh Token dựa trên tiêu chuẩn JWT để duy trì phiên làm việc liên tục cho ứng dụng phía client. Việc sử dụng JWT cho phép hệ thống duy trì trạng thái đăng nhập một cách hiệu quả cả trên các giao thức HTTP truyền thống lẫn các kết nối WebSocket thời gian thực.

Cuối cùng, chức năng đăng xuất được xây dựng để đảm bảo tính an toàn tuyệt đối cho phiên làm việc sau khi kết thúc. Khi người dùng chủ động đăng xuất, hệ thống triển khai cơ chế danh sách đen (blacklist) để vô hiệu hóa ngay lập tức mã truy cập hiện tại, ngăn chặn mọi nỗ lực sử dụng lại token trái phép. Đồng thời, các mã làm mới (Refresh Token) lưu trữ trong cơ sở dữ liệu cũng sẽ bị xóa bỏ hoàn toàn, đảm bảo rằng mọi quyền truy cập liên quan đến tài khoản đều bị thu hồi dứt điểm khi người dùng không còn duy trì phiên làm việc.

1.5.2. Quản lý tài liệu

Chức năng quản lý tài liệu đóng vai trò là xương sống trong toàn bộ hệ thống, cung cấp cho người dùng một môi trường làm việc có cấu trúc để tổ chức và vận hành các tệp văn bản cá nhân. Người dùng được trang bị đầy đủ các thao tác cơ bản phục vụ trọn vẹn vòng đời của một tài liệu bao gồm việc khởi tạo tệp mới, mở các tệp hiện có, thay đổi tiêu đề tài liệu và xóa bỏ các tệp tin khi không còn nhu cầu sử dụng. Để đảm bảo sự thuận tiện và trực quan trong quá trình làm việc, hệ thống tích hợp giao diện bảng điều khiển trung tâm, nơi hiển thị danh sách toàn bộ các tài liệu mà người dùng sở hữu hoặc được người khác cấp quyền truy cập, kèm theo các thông tin chi tiết về ngày khởi tạo và thời điểm cập nhật gần nhất.

Nhằm giải quyết triệt để rủi ro mất mát dữ liệu do các sự cố ngoài ý muốn như mất kết nối mạng bất ngờ hay trình duyệt gặp lỗi kỹ thuật, hệ thống tích hợp cơ chế tự động lưu trữ bản nháp (autosave). Cơ chế này hoạt động hoàn toàn tự động ở chế độ nền, thực hiện ghi nhận nội dung văn bản vào cơ sở dữ liệu theo chu kỳ định kỳ từ 5 đến 10 giây hoặc ngay khi hệ thống phát hiện hành vi người dùng sắp rời khỏi giao diện soạn thảo. Việc lưu trữ không chỉ dừng lại ở văn bản thuần túy mà còn đồng bộ hóa đồng thời các định dạng JSON, HTML và trạng thái nhị phân của Yjs, đảm bảo toàn bộ công sức soạn thảo được bảo vệ liên tục và an toàn tuyệt đối.

Cấu trúc lưu trữ tài liệu trong hệ thống được thiết kế chuyên biệt để tối ưu hóa cho từng mục đích sử dụng trong môi trường phân tán. Trong cơ sở dữ liệu MySQL, nội dung của mỗi tài liệu được phân tách rõ ràng thành các thành phần dữ liệu riêng biệt: `content_json` đóng vai trò là nguồn chân lý cho định dạng rich text, `content_text` phục vụ cho các tác vụ tìm kiếm nhanh, `content_html` hỗ trợ hiển thị nội dung trên trình duyệt và `ydoc_state` để tái thiết lập chính xác môi trường làm việc cộng tác. Việc tách biệt dữ liệu theo cách này giúp hệ thống đạt hiệu suất cao trong cả việc truy xuất thông tin tức thời lẫn duy trì trạng thái đồng bộ phức tạp.

Cuối cùng, hệ thống cung cấp chức năng tạo bản sao tài liệu (cloning) để tăng cường tính linh hoạt trong quản lý công việc cho người dùng. Khi thực hiện sao chép, hệ thống sẽ tạo ra một bản ghi mới với ID định danh riêng biệt, kế thừa toàn bộ nội dung của tài liệu gốc tại thời điểm sao chép nhưng loại bỏ danh sách cộng tác viên cũ nhằm đảm bảo tính bảo mật và quyền riêng tư. Người dùng thực hiện lệnh sao chép sẽ

mặc định trở thành chủ sở hữu của tài liệu mới này, cho phép họ chủ động thiết lập lại quyền truy cập hoặc điều chỉnh nội dung hoàn toàn độc lập mà không làm ảnh hưởng tới bất kỳ cấu trúc hay lịch sử nào của tài liệu gốc.

1.5.3. Soạn thảo rich text

Hệ thống cung cấp một môi trường soạn thảo văn bản chuyên sâu thông qua việc tích hợp framework TipTap, cho phép người dùng thực hiện các thao tác định dạng văn bản giàu (rich text) một cách trực quan và hiệu quả. Các tiện ích mở rộng được cấu hình sẵn bao gồm StarterKit cho các định dạng cơ bản, cùng với các nhóm công cụ chuyên biệt như hỗ trợ gạch chân, căn lề, tạo liên kết, tùy chỉnh kiểu chữ, màu sắc và làm nổi bật văn bản bằng các hiệu ứng highlight. Sự kết hợp này đảm bảo người dùng có đầy đủ các công cụ cần thiết để trình bày văn bản theo tiêu chuẩn chuyên nghiệp ngay trên nền tảng web mà không cần thông qua các phần mềm biên tập trung gian.

Để tối ưu hóa trải nghiệm người dùng trong môi trường cộng tác nhiều người, tính năng hoàn tác và làm lại (Undo/Redo) được thiết kế theo cơ chế cục bộ thông qua trình quản lý Y.UndoManager. Cơ chế này đảm bảo rằng các thao tác quay lại hoặc khôi phục trạng thái chỉ có hiệu lực đối với chuỗi hành động của chính người dùng đó trong phiên làm việc hiện tại. Việc giới hạn phạm vi tác động của chức năng này giúp ngăn chặn triệt để tình trạng một người dùng vô tình làm mất hoặc ghi đè lên nội dung do các cộng tác viên khác vừa thực hiện, từ đó duy trì sự ổn định cho tài liệu trong suốt quá trình làm việc nhóm.

1.5.4. Cộng tác thời gian thực

Tính năng cộng tác thời gian thực là điểm nhấn công nghệ quan trọng nhất của hệ thống, đòi hỏi sự phối hợp nhịp nhàng giữa hạ tầng máy chủ và thuật toán xử lý dữ liệu phân tán để đảm bảo trải nghiệm liền mạch cho người dùng. Hệ thống áp dụng cấu trúc dữ liệu CRDT (Conflict-free Replicated Data Type) thông qua thư viện Yjs để giải quyết triệt để bài toán xung đột dữ liệu khi nhiều người dùng đồng thời chỉnh sửa một tài liệu từ các thiết bị khác nhau. Thay vì truyền tải toàn bộ cấu trúc văn bản sau mỗi phím gõ gây lãng phí băng thông, hệ thống chỉ gửi đi các gói dữ liệu cập nhật nhị phân siêu nhỏ, qua đó đảm bảo độ trễ thấp và tính nhất quán cuối cùng trên tất cả

các trình duyệt client. Mọi thay đổi đều được máy chủ tiếp nhận, xác thực quyền hạn, lưu vết vào lịch sử và phát quảng bá tới tất cả các thành viên trong cùng phòng ảo, giúp mọi người đều thấy nội dung thay đổi gần như ngay lập tức.

Bên cạnh việc đồng bộ nội dung, hệ thống còn chú trọng đến yếu tố nhận thức không gian nhằm nâng cao hiệu quả phối hợp giữa các thành viên. Thông qua module CollaborationCursor kết hợp cùng giao thức WebSocket của Socket.IO, hệ thống cho phép người dùng quan sát trực quan vị trí con trỏ chuột, nhãn tên định danh và vùng văn bản đang được các cộng tác viên khác bôi đen lựa chọn trong thời gian thực. Các thông tin về trạng thái tương tác này được truyền tải dưới dạng sự kiện mạng tạm thời, không lưu trữ bền vững vào cơ sở dữ liệu, giúp tối ưu hóa tài nguyên hệ thống đồng thời tạo cảm giác gần gũi như đang làm việc trực tiếp bên cạnh nhau trong cùng một không gian văn phòng ảo.

1.5.5. Phân quyền tài liệu

Hệ thống thiết lập cơ chế kiểm soát truy cập phân tầng nhằm bảo vệ tính toàn vẹn và quyền riêng tư của tài liệu trong quá trình chia sẻ dữ liệu cộng tác. Người dùng được phân chia vào ba nhóm vai trò chính gồm chủ sở hữu, biên tập viên và người xem, với mỗi vai trò sở hữu các quyền hạn được quy định nghiêm ngặt để đảm bảo an toàn thông tin. Chủ sở hữu là người khởi tạo tài liệu và được trao toàn quyền bao gồm việc xem, sửa, xóa nội dung, mời thêm cộng tác viên mới, tùy chỉnh cấp bậc quyền hạn của thành viên khác và quyền khôi phục lịch sử các phiên bản đã lưu.

Vai trò biên tập viên được cấp đặc quyền xem, chỉnh sửa nội dung văn bản để phối hợp làm việc và có quyền khôi phục (restore) lại các phiên bản cũ của tài liệu. Tuy nhiên, họ bị hạn chế đối với các chức năng quản trị cấp cao như xóa tài liệu hay quản lý phân quyền thành viên nhằm tránh những thay đổi cấu trúc không mong muốn.

Vai trò người xem được thiết kế với quyền hạn tối thiểu, chỉ cho phép tiếp cận thông tin để đọc và tham khảo, mọi nỗ lực can thiệp vào cấu trúc tài liệu từ nhóm này đều bị hệ thống chặn đứng. Để thực thi chính sách này một cách nhất quán, backend được cấu hình kiểm tra chặt chẽ vai trò của người dùng trong bảng `document_collaborators` trước khi chấp nhận bất kỳ yêu cầu thao tác nào qua API

hoặc WebSocket. Đối với các tài khoản chỉ có quyền xem, giao diện frontend sẽ tự động ẩn hoặc khóa toàn bộ các thanh công cụ định dạng và tính năng chỉnh sửa, đồng thời máy chủ sẽ từ chối mọi gói tin cập nhật (update) được gửi lên từ phía client, đảm bảo rằng chỉ những người dùng đủ thẩm quyền mới có khả năng tạo ra các thay đổi mang tính bền vững trên tài liệu.

1.5.6. Lịch sử phiên bản

Chức năng quản lý lịch sử phiên bản đóng vai trò như một cơ chế bảo hiểm dữ liệu quan trọng, cho phép hệ thống lưu trữ và tái hiện lại các mốc trạng thái tài liệu theo thời gian. Hệ thống hỗ trợ việc tạo các bản chụp nhanh (snapshot) phiên bản thủ công hoặc tự động, trong đó mỗi bản lưu trữ phải chứa đựng đầy đủ các định dạng biểu diễn dữ liệu cần thiết như văn bản thô, mã HTML, cấu trúc JSON và đặc biệt là trạng thái nhị phân của Yjs. Việc lưu trữ đồng bộ này cho phép hệ thống không chỉ ghi nhận nội dung văn bản mà còn bảo toàn được toàn bộ cấu trúc định dạng giàu (rich text) phức tạp, giúp người dùng có thể xem lại hoặc phục hồi chính xác môi trường cộng tác của bất kỳ thời điểm nào trong quá khứ.

Khi có nhu cầu khôi phục, hệ thống cung cấp một giao diện quản trị phiên bản chuyên biệt, nơi người dùng có vai trò chủ sở hữu hoặc biên tập viên có thể truy xuất danh sách các mốc lịch sử đã được lưu trữ trước đó. Người dùng có thể thực hiện thao tác xem chi tiết để đối chiếu nội dung trước khi quyết định thực hiện lệnh khôi phục toàn diện. Ngay khi lệnh khôi phục được xác nhận, máy chủ sẽ tiến hành cập nhật lại dữ liệu tài liệu hiện tại với nội dung của phiên bản cũ, đồng thời thực hiện tăng `current_version` và cập nhật `ydoc_state` sang snapshot của phiên bản đã chọn, sau đó broadcast trạng thái mới này đến tất cả các máy trạm đang mở tài liệu để đảm bảo sự đồng nhất tuyệt đối cho tất cả thành viên trong phòng cộng tác.

1.5.7. Tạo bản sao tài liệu

Chức năng tạo bản sao tài liệu được thiết kế nhằm mang lại sự linh hoạt tối đa cho người dùng trong việc quản lý vòng đời dữ liệu, cho phép khởi tạo một bản sao hoàn toàn độc lập từ một tệp tin gốc có sẵn. Khi người dùng thực hiện yêu cầu sao chép, backend sẽ tiến hành xác minh quyền truy cập để đảm bảo tính bảo mật, đồng thời thực hiện sao chép toàn bộ các thành phần dữ liệu bao gồm văn bản thuần túy,

định dạng JSON, mã HTML và trạng thái nhị phân của Yjs sang một bản ghi mới có định danh ID riêng biệt. Người thực hiện lệnh sao chép sẽ mặc định được gán quyền chủ sở hữu (owner) đối với tài liệu mới, còn toàn bộ danh sách thành viên cộng tác của tài liệu gốc sẽ bị loại bỏ hoàn toàn để đảm bảo tính riêng tư và bảo mật thông tin. Ngoài ra, nếu người dùng để trống phần tiêu đề khi tạo bản sao, hệ thống sẽ tự động cấu trúc tên mới bằng cách bổ sung thêm hậu tố ghi chú vào sau tên của tài liệu gốc giúp người dùng dễ dàng phân biệt và quản lý danh mục tài liệu của mình.

1.6. Phân tích yêu cầu phi chức năng

1.6.1. Bảo mật

Hệ thống đặt ưu tiên hàng đầu vào công tác bảo mật thông tin người dùng và đảm bảo tính toàn vẹn của dữ liệu trong môi trường làm việc phân tán. Đối với thông tin định danh, toàn bộ mật khẩu của người dùng bắt buộc phải được xử lý qua thuật toán băm (hashing) an toàn trước khi lưu trữ chính thức vào cơ sở dữ liệu, đảm bảo rằng ngay cả khi xảy ra sự cố lộ lọt dữ liệu thì thông tin gốc cũng không bị phơi bày cho các đối tượng xấu.

Mọi kết nối giữa ứng dụng khách và máy chủ, bao gồm cả các truy vấn qua REST API và kết nối WebSocket, đều được bảo vệ nghiêm ngặt thông qua việc xác thực mã thông báo JWT. Hệ thống thực hiện kiểm tra quyền hạn chặt chẽ dựa trên vai trò của người dùng như owner, editor, hay viewer đối với từng tài liệu cụ thể ngay tại tầng xử lý nghiệp vụ của backend. Điều này được thực hiện trước khi chấp nhận bất kỳ yêu cầu thao tác nào từ phía client, đảm bảo tính bảo mật trong toàn bộ quy trình vận hành.

Cơ chế này ngăn chặn hiệu quả các hành vi truy cập trái phép hoặc chỉnh sửa tài liệu ngoài thẩm quyền, ngay cả khi các đối tượng cố tình gửi yêu cầu thao tác sai lệch với vai trò thực tế của họ. Thông qua việc kết hợp giữa xác thực JWT và kiểm duyệt quyền tại tầng service, hệ thống bảo vệ tài liệu một cách toàn diện, loại bỏ nguy cơ từ các tài khoản không có quyền hạn hợp lệ và duy trì môi trường làm việc nhóm an toàn.

1.6.2. Tính nhất quán dữ liệu

Hệ thống được thiết kế để đảm bảo tính nhất quán dữ liệu trong môi trường phân tán, nơi nhiều người dùng có thể thực hiện thao tác chỉnh sửa cùng lúc trên cùng một tài liệu. Để giải quyết bài toán xung đột này, hệ thống áp dụng cơ chế CRDT (Conflict-free Replicated Data Type) thông qua thư viện Yjs, cho phép tự động hợp nhất các thay đổi mà không cần sự can thiệp từ máy chủ trung gian.

Mỗi khi một người dùng thực hiện thay đổi, Yjs sẽ tạo ra các gói cập nhật nhỏ phân siêu nhỏ thay vì truyền tải toàn bộ cấu trúc văn bản, giúp tối ưu hóa hiệu suất truyền tải trên mạng. Khi các gói tin này được gửi qua Socket.IO, chúng được áp dụng vào cấu trúc dữ liệu Y.Doc của các client khác, đảm bảo tất cả các bên đều hội tụ về một trạng thái cuối cùng giống nhau bất kể thứ tự nhận gói tin.

Bên cạnh đó, hệ thống duy trì tính nhất quán bền vững thông qua việc lưu trữ đa định dạng tại cơ sở dữ liệu MySQL. Các trường dữ liệu như `content_json`, `content_html` và `ydoc_state` được cập nhật định kỳ hoặc khi có sự kiện quan trọng, đóng vai trò là điểm tham chiếu tin cậy. Sự kết hợp giữa cơ chế CRDT thời gian thực và việc lưu trữ snapshot định dạng giàu giúp hệ thống đảm bảo cả tính nhất quán tức thời cho người dùng và tính nhất quán lâu dài khi tải lại tài liệu.

1.6.3. Khả năng mở rộng

Khả năng mở rộng của hệ thống được xây dựng trên nền tảng kiến trúc Client-Server với sự phân tách rõ ràng giữa các lớp, tạo nền tảng vững chắc cho việc phát triển và nâng cấp lâu dài. Việc phân tách rãnh mạch giữa giao diện frontend dựa trên React và hệ thống backend vận hành bằng Node.js kết hợp Express cho phép đội ngũ phát triển thực hiện các thay đổi, tối ưu hóa hoặc nâng cấp từng thành phần độc lập mà không gây ảnh hưởng tiêu cực đến toàn bộ cấu trúc hệ thống.

Backend của hệ thống được tổ chức theo kiến trúc phân tầng domain, trong đó các logic nghiệp vụ được đóng gói nghiêm ngặt trong từng module riêng biệt như quản lý xác thực, người dùng và tài liệu. Cấu trúc này không chỉ giúp mã nguồn trở nên minh bạch, dễ dàng kiểm thử mà còn cho phép nhóm phát triển linh hoạt tích hợp thêm các tính năng mới trong tương lai hoặc thay thế các dịch vụ xử lý hiện hữu mà không làm xáo trộn luồng logic nghiệp vụ cốt lõi đã được định nghĩa.

Việc ứng dụng Sequelize ORM để làm cầu nối tương tác với cơ sở dữ liệu MySQL cũng đóng vai trò chiến lược trong việc đảm bảo tính mở rộng. Bằng cách quản lý các quan hệ dữ liệu phức tạp thông qua các model JavaScript thay vì các câu lệnh SQL viết tay, hệ thống có khả năng thích nghi nhanh chóng với những thay đổi về lược đồ cơ sở dữ liệu khi nhu cầu nghiệp vụ tăng lên, đồng thời hỗ trợ tốt cho việc chuyển đổi hoặc mở rộng sang các hệ quản trị cơ sở dữ liệu khác khi cần thiết.

1.6.4. Khả năng phục hồi khi mất kết nối

Khả năng phục hồi khi mất kết nối (offline editing) là một yêu cầu phi chức năng thiết yếu để đảm bảo trải nghiệm người dùng không bị gián đoạn trong các tình huống mạng không ổn định. Hệ thống cho phép người dùng tiếp tục soạn thảo nội dung văn bản bình thường ngay cả khi không có kết nối internet, nhờ vào việc thư viện Yjs tự động tạo ra các bản cập nhật cục bộ và lưu trữ trạng thái vào IndexedDB ngay trên trình duyệt của người dùng.

Khi kết nối mạng được khôi phục, client sẽ tự động thiết lập lại liên kết với máy chủ thông qua Socket.IO và gửi các thay đổi còn tồn đọng (pending updates) lên hệ thống. Máy chủ sau đó thực hiện quá trình hợp nhất các cập nhật này với trạng thái hiện tại của tài liệu bằng thuật toán CRDT, đảm bảo mọi dữ liệu soạn thảo khi ngoại tuyến được tích hợp một cách an toàn và nhất quán với nội dung của các cộng tác viên khác.

Luồng đồng bộ này được quản lý chặt chẽ thông qua các sự kiện mạng đặc thù như reconnect-sync và sync-completed, giúp người dùng luôn nắm bắt được trạng thái đồng bộ của tài liệu. Điều này không chỉ đảm bảo dữ liệu luôn được an toàn mà còn tạo cảm giác tin cậy, khẳng định tính sẵn sàng cao của hệ thống ngay cả trong những điều kiện mạng không ổn định.

CHƯƠNG 2. THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

2.1. Kiến trúc tổng thể hệ thống

Hệ thống soạn thảo văn bản cộng tác được thiết kế theo mô hình Client - Server kết hợp với cơ chế đồng bộ dữ liệu thời gian thực và xử lý xung đột trong hệ thống phân tán. Kiến trúc của hệ thống được chia thành ba thành phần chính với các chức năng và luồng tương tác rõ ràng

2.1.1. Lớp giao diện người dùng (Frontend / Client)

- Nền tảng công nghệ: Giao diện người dùng được phát triển dựa trên thư viện React và sử dụng công cụ build Vite.
- Công cụ soạn thảo: Hệ thống tích hợp TipTap làm công cụ soạn thảo văn bản giàu định dạng (Rich Text Editor), kết hợp cùng thư viện Yjs để hỗ trợ tính năng chỉnh sửa cộng tác.
- Kết nối: Client sử dụng Socket.IO Client để thiết lập và duy trì luồng kết nối dữ liệu liên tục với máy chủ.

2.1.2. Lớp xử lý nghiệp vụ (Backend / Server)

- Nền tảng công nghệ: Backend được xây dựng trên môi trường Node.js kết hợp cùng framework Express.
- Cấu trúc phân tầng: Hệ thống backend được tổ chức theo module/domain (bao gồm auth, user, document) với cấu trúc các lớp riêng biệt bao gồm: routes -> controllers -> services -> repositories -> Sequelize models -> MySQL. Cấu trúc này giúp phân tách rõ ràng giữa định tuyến, kiểm soát request, logic nghiệp vụ và tương tác với cơ sở dữ liệu.
- Giao tiếp HTTP và WebSocket: Server cung cấp hai luồng giao tiếp song song. HTTP API được dùng cho các thao tác chuẩn (CRUD, quản lý phiên bản, kiểm tra quyền truy cập), trong khi WebSocket (thông qua Socket.IO) đảm nhận nhiệm vụ đồng bộ cập nhật chỉnh sửa theo thời gian thực giữa những người dùng (realtime sync).

2.1.3. Lớp lưu trữ dữ liệu (Database / Storage)

- Hệ thống lưu trữ sử dụng hệ quản trị cơ sở dữ liệu MySQL và tương tác thông qua Sequelize ORM.
- Để đáp ứng nhu cầu tái hiện tài liệu linh hoạt và đồng bộ theo thời gian thực, hệ thống lưu trữ nhiều định dạng dữ liệu khác nhau cho một văn bản, bao gồm content_text, content_json, content_html và đặc biệt là ydoc_state - trạng thái nhị phân cốt lõi phục vụ xử lý xung đột tài liệu.

2.1.4. Luồng xử lý và cơ chế đồng bộ (Synchronization)

- Để xử lý việc nhiều người dùng tác động lên tài liệu cùng lúc mà không làm hỏng dữ liệu, hệ thống áp dụng cơ chế Yjs/CRDT (Conflict-free Replicated Data Type).
- Khi người dùng thao tác, Client tạo ra các "Yjs update" và truyền đi thông qua Socket.IO.
- Server (tại tầng service) sẽ tiếp nhận các cập nhật này, kiểm tra quyền hạn (chỉ các tài khoản có vai trò owner hoặc editor mới được cấp phép thực hiện), sau đó lưu sự thay đổi vào CSDL (document_updates), tự động gộp (merge) để giải quyết xung đột vào ydoc_state và cuối cùng phát (broadcast) lại bản cập nhật đó cho những người dùng khác trong cùng phiên làm việc để đạt tính nhất quán cuối cùng (eventual consistency).

2.2. Thiết kế backend

Hệ thống backend được xây dựng trên nền tảng Node.js kết hợp với framework Express, sử dụng Sequelize ORM để tương tác với cơ sở dữ liệu MySQL. Mã nguồn được tổ chức theo mô hình phân lớp rõ ràng, hướng tới mục tiêu dễ bảo trì và dễ mở rộng trong quá trình phát triển.

2.2.1. Kiến trúc module backend

Mã nguồn backend được chia theo từng module hoặc domain cụ thể, bao gồm các domain chính như auth (xác thực), user (người dùng) và document (tài liệu). Trong mỗi module, hệ thống áp dụng thống nhất một cấu trúc phân lớp luồng dữ liệu

một chiều nghiêm ngặt: routes -> controllers -> services -> repositories -> Sequelize models -> MySQL.

2.2.2. Luồng xử lý API

Hệ thống quy định rõ vai trò và trách nhiệm của từng lớp thành phần như sau:

- Routes: Chỉ có nhiệm vụ khai báo các endpoint API và gắn các middleware tương ứng.
- Controllers: Tiếp nhận request từ phía client, gọi các phương thức ở tầng service và định dạng response trả về.
- Services: Đảm nhận toàn bộ quá trình xử lý nghiệp vụ, kiểm tra quyền truy cập; đặc biệt, lớp này tuyệt đối không chứa các câu lệnh truy vấn cơ sở dữ liệu (SQL). Cả cập nhật qua HTTP và WebSocket đều phải đi qua tầng này để kiểm duyệt.
- Repositories: Là tầng duy nhất chịu trách nhiệm thao tác trực tiếp với cơ sở dữ liệu thông qua các model của Sequelize.

2.2.3. Xác thực người dùng bằng JWT

Hệ thống sử dụng cơ chế JSON Web Token (JWT) cho cả REST API và WebSocket.

- Khi người dùng thực hiện đăng nhập bằng tài khoản và mật khẩu hợp lệ, hệ thống sẽ mã hóa và cấp phát đồng thời hai loại mã thông báo: một mã truy cập (access token) có thời hạn ngắn phục vụ đính kèm vào tiêu đề của mọi yêu cầu gửi lên máy chủ và một mã làm mới (refresh token) lưu trữ an toàn để gia hạn phiên làm việc khi mã cũ hết hạn.
- Nhằm tối ưu hóa tính an toàn khi người dùng chủ động đăng xuất, hệ thống triển khai cơ chế danh sách đen (blacklist). Khi nhận tín hiệu đăng xuất, hệ thống ngay lập tức đưa mã truy cập hiện tại vào danh sách đen để chặn mọi quyền sử dụng mã này trong tương lai, đồng thời xóa hoàn toàn mã làm mới khỏi hệ thống lưu trữ lưu vết phiên.

2.2.4. Phân quyền owner, editor, viewer

Cơ chế kiểm soát truy cập phân chia người dùng thành ba vai trò (role) trên mỗi tài liệu:

Bảng 2.1. Bảng phân quyền người dùng

Role	Xem	Sửa	Xóa document	Mời user	Đổi quyền	Restore version
owner	Có	Có	Có	Có	Có	Có
editor	Có	Có	Không	Không	Không	Có
viewer	Có	Không	Không	Không	Không	Không

2.2.5. Thiết kế REST API

Giao tiếp HTTP giữa máy chủ và các ứng dụng biên tuân thủ các quy chuẩn thiết kế kiến trúc RESTful nhằm đảm bảo tính tường minh, kết hợp công cụ tự động tạo tài liệu kiểm thử để hỗ trợ nhà phát triển. Các nhóm API nghiệp vụ cốt lõi bao gồm:

- **Nhóm xác thực hệ thống:** Bao gồm các cổng đăng ký thành viên mới, đăng nhập tài khoản để nhận cặp mã bảo mật, cổng làm mới phiên làm việc định kỳ và cổng kiểm tra thông tin định danh của người dùng hiện tại.
- **Nhóm quản lý cấu trúc tài liệu:** Cung cấp các thao tác lấy danh sách tài liệu tổng quan, khởi tạo tài liệu mới chỉ với tên tiêu đề, mở chi tiết một văn bản để nhận về cấu trúc nội dung đa định dạng (văn bản thô, định dạng JSON, mã HTML hiển thị nhanh, khối nhị phân trạng thái cộng tác) và xóa tài liệu dành riêng cho chủ sở hữu. Đặc biệt, hệ thống cung cấp API lưu trữ ảnh chụp nhanh (snapshot) nội dung định dạng giàu, cập nhật song song cấu trúc giao diện và văn bản thô phục vụ tìm kiếm.
- **Nhóm quản lý thành viên cộng tác:** Cung cấp các phương thức truy vấn danh sách những người tham gia, mời thành viên mới thông qua định danh thư điện tử, cập nhật nâng hoặc hạ cấp bậc quyền hạn giữa hai vai trò biên tập viên hoặc người xem, và thu hồi quyền truy cập của một thành viên.
- **Nhóm quản lý dòng thời gian phiên bản:** Cho phép lập danh sách tất cả các điểm mốc lịch sử, tạo mới một mốc phiên bản chứa đầy đủ các biểu diễn dữ liệu cấu trúc và nhị phân tại thời điểm lưu, xem chi tiết dữ liệu quá khứ và thực hiện lệnh khôi phục toàn diện.

Đối với luồng nghiệp vụ tạo bản sao tài liệu, hệ thống thiết kế một quy trình xử lý khép kín tại tầng nghiệp vụ để bảo vệ an toàn thông tin: Khi người dùng gửi yêu cầu sao chép một văn bản cụ thể, hệ thống trước hết sẽ xác minh tài khoản đó có tối thiểu quyền đọc tài liệu gốc. Nếu hợp lệ, tầng lưu trữ thực hiện sao chép toàn bộ nội dung ảnh chụp hiện tại sang một bản ghi tài liệu hoàn toàn mới trong cơ sở dữ liệu. Người khởi tạo yêu cầu sẽ tự động được hệ thống gán vai trò chủ sở hữu đối với tài liệu mới này. Nhằm đảm bảo tính riêng tư, toàn bộ danh sách các cộng tác viên cũ thuộc tài liệu gốc sẽ bị loại bỏ hoàn toàn khỏi bản sao mới. Trong trường hợp người dùng để trống phần tiêu đề bản sao, hệ thống sẽ tự động cấu trúc tên theo cú pháp bổ sung hậu tố ghi chú sao chép vào sau tên tài liệu gốc.

2.2.6. Thiết kế Socket.IO backend

Kênh giao tiếp qua giao thức kết nối WebSocket, sử dụng thư viện Socket.IO, là hạ tầng cốt lõi phục vụ truyền tải và xử lý đồng bộ thời gian thực. Để ngăn chặn các cuộc tấn công mạng, mọi yêu cầu thiết lập kết nối từ phía các ứng dụng khách bắt buộc phải truyền kèm mã truy cập bảo mật trong cấu hình xác thực. Máy chủ tiến hành giải mã, đối chiếu trạng thái hoạt động của mã với danh sách đen, và gán thông tin định danh của người dùng trực tiếp vào dữ liệu phiên kết nối để quản lý xuyên suốt chu kỳ sống của socket.

Hệ thống thiết kế luồng xử lý và đồng bộ thông qua các sự kiện mạng đặc thù sau:

- **Sự kiện tham gia phòng tài liệu:** Khi client phát tín hiệu tham gia vào một tài liệu, máy chủ tiến hành kiểm tra quyền đọc của tài khoản. Khi xác thực thành công, máy chủ đưa kết nối mạng này vào một không gian phòng ảo độc lập được định danh theo mã tài liệu, ghi nhớ quyền truy cập vào bộ nhớ đệm kết nối để tối ưu tốc độ xử lý cho các thao tác sau, đồng thời phát thông báo danh sách thành viên đang trực tuyến đến toàn phòng. Khi người dùng chủ động rời đi hoặc ngắt kết nối đột ngột, hệ thống tự động loại bỏ kết nối khỏi phòng ảo và cập nhật lại danh sách trực tuyến.
- **Luồng xử lý đồng bộ chỉnh sửa:** Nhằm tối ưu hóa băng thông mạng, hệ thống tuyệt đối không truyền tải lại toàn bộ cấu trúc văn bản sau mỗi phím gõ của

người dùng. Khi có thao tác chỉnh sửa nội dung hoặc thay đổi định dạng, client chỉ gửi đi các gói dữ liệu cập nhật nhị phân siêu nhỏ được tạo ra bởi thuật toán phân tán. Khi tiếp nhận gói cập nhật, bộ xử lý của máy chủ lập tức gọi tăng nghiệp vụ để xác thực quyền hạn, yêu cầu người gửi bắt buộc phải giữ vai trò chủ sở hữu hoặc biên tập viên. Nếu một tài khoản chỉ có quyền xem cổ tình tự phát tín hiệu cập nhật, máy chủ sẽ chặn lại và phản hồi thông báo từ chối quyền. Sau khi vượt qua vòng kiểm duyệt, tăng lưu trữ sẽ ghi lại gói thay đổi vào bảng lịch sử cập nhật và thực hiện gộp trực tiếp dữ liệu nhị phân này vào trạng thái tài liệu gốc trong cơ sở dữ liệu. Cuối cùng, máy chủ phát quảng bá gói cập nhật này đến tất cả các kết nối khác trong cùng phòng ảo để cập nhật giao diện hiển thị đồng nhất.

- **Cơ chế phục hồi sau mất kết nối:** Để hỗ trợ tính năng làm việc ngoại tuyến, khi một ứng dụng khách kết nối lại sau khoảng thời gian mất mạng, nó sẽ phát một yêu cầu đồng bộ kèm theo một vector trạng thái đại diện cho mốc dữ liệu cục bộ hiện tại. Máy chủ tiếp nhận vector này, đối chiếu với trạng thái nhị phân đầy đủ đang lưu trữ tại cơ sở dữ liệu để tính toán chính xác phần dữ liệu thiếu hụt mà client chưa nhận được, sau đó đóng gói và phản hồi lại đoạn cập nhật cần bù đắp, đưa hai bên về trạng thái nhất quán cuối cùng.
- **Luồng truyền tải nhận thức không gian:** Các thông tin mang tính chất tạm thời, thay đổi liên tục như tọa độ con trỏ chuột, định danh vùng văn bản đang được bôi đen và thuộc tính màu sắc cá nhân được truyền tải qua các sự kiện cập nhật nhận thức. Nhằm tiết kiệm tài nguyên xử lý của máy chủ, các gói tin này chỉ được phép phân phát quảng bá khi và chỉ khi kết nối gửi tin đã hoàn tất thủ tục gia nhập và đang nằm trong phòng ảo của tài liệu đó. Máy chủ không thực hiện lưu vết các dữ liệu nhận thức này vào cơ sở dữ liệu bền vững.

2.3. Thiết kế frontend

Để đẩy nhanh tốc độ thiết kế, nhóm đã sử dụng AI stitch.withgoogle.com để thiết kế toàn bộ. Nhóm xây dựng prompt từ các chức năng có sẵn và nhập câu lệnh để Stich sinh ra giao diện, sau đó tùy chỉnh lại một số phần cho phù hợp. Dưới đây là kết quả đạt được sau khi thiết kế

2.3.1. Cấu trúc giao diện React

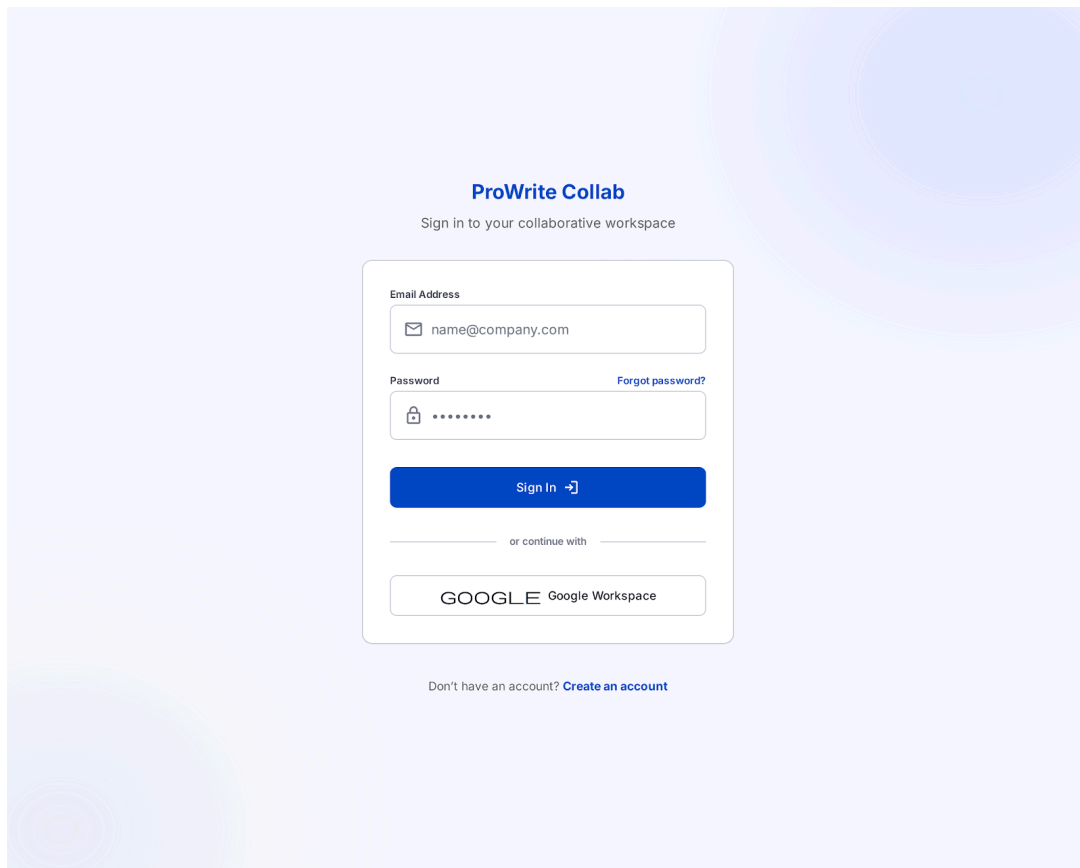
Hệ thống được xây dựng theo mô hình ứng dụng web đơn trang (Single Page Application – SPA) sử dụng React ở phía frontend. Giao diện được tổ chức theo hướng phân tách thành các trang chức năng độc lập kết hợp với cơ chế điều hướng giữa các trang nhằm tạo trải nghiệm liền mạch cho người dùng. Mỗi trang đảm nhiệm một vai trò riêng trong hệ thống và được kết nối thông qua các luồng thao tác thống nhất.

Nhóm xây dựng các trang chính bao gồm:

- Trang đăng nhập
- Trang đăng ký
- Trang danh sách tài liệu
- Trang soạn thảo tài liệu
- Toolbar rich text
- Trang lịch sử phiên bản

Các trang trên được thiết kế theo kiến trúc component nhằm tăng khả năng tái sử dụng, dễ bảo trì và thuận tiện mở rộng chức năng trong tương lai. Những thành phần giao diện dùng chung như thanh điều hướng, hộp thoại xác nhận, thanh công cụ định dạng, menu người dùng và các thành phần hiển thị tài liệu được tách thành các component độc lập.

2.3.2. Trang đăng nhập và đăng ký



The image shows a login and registration page for 'ProWrite Collab'. The page has a light blue background with a subtle pattern of overlapping circles. At the top center, the text 'ProWrite Collab' is displayed in a bold, dark blue font. Below it, the subtitle 'Sign in to your collaborative workspace' is in a smaller, gray font. The main content is a white rectangular box with rounded corners. Inside this box, there are two input fields: 'Email Address' and 'Password'. The 'Email Address' field contains the text 'name@company.com' and has an envelope icon on the left. The 'Password' field contains a series of dots and has a lock icon on the left. To the right of the password field, there is a link that says 'Forgot password?'. Below the input fields is a blue button with the text 'Sign In' and a right-pointing arrow. Below the button, there is a horizontal line with the text 'or continue with' in the center. Below this line is a button with the Google logo and the text 'Google Workspace'. At the bottom of the white box, there is a link that says 'Don't have an account? Create an account'.

ProWrite Collab

Sign in to your collaborative workspace

Email Address

name@company.com

Password

Forgot password?

Sign In →

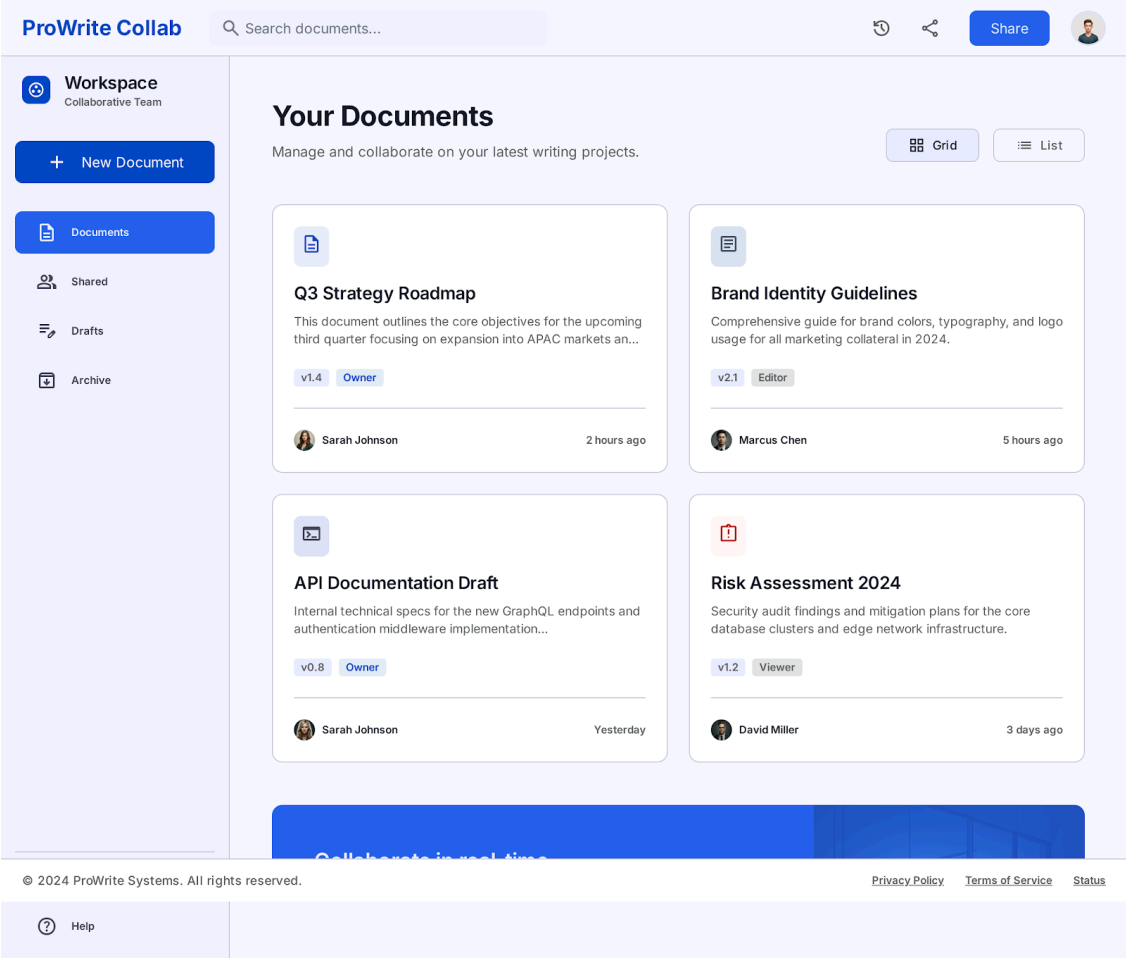
or continue with

GOOGLE Google Workspace

Don't have an account? [Create an account](#)

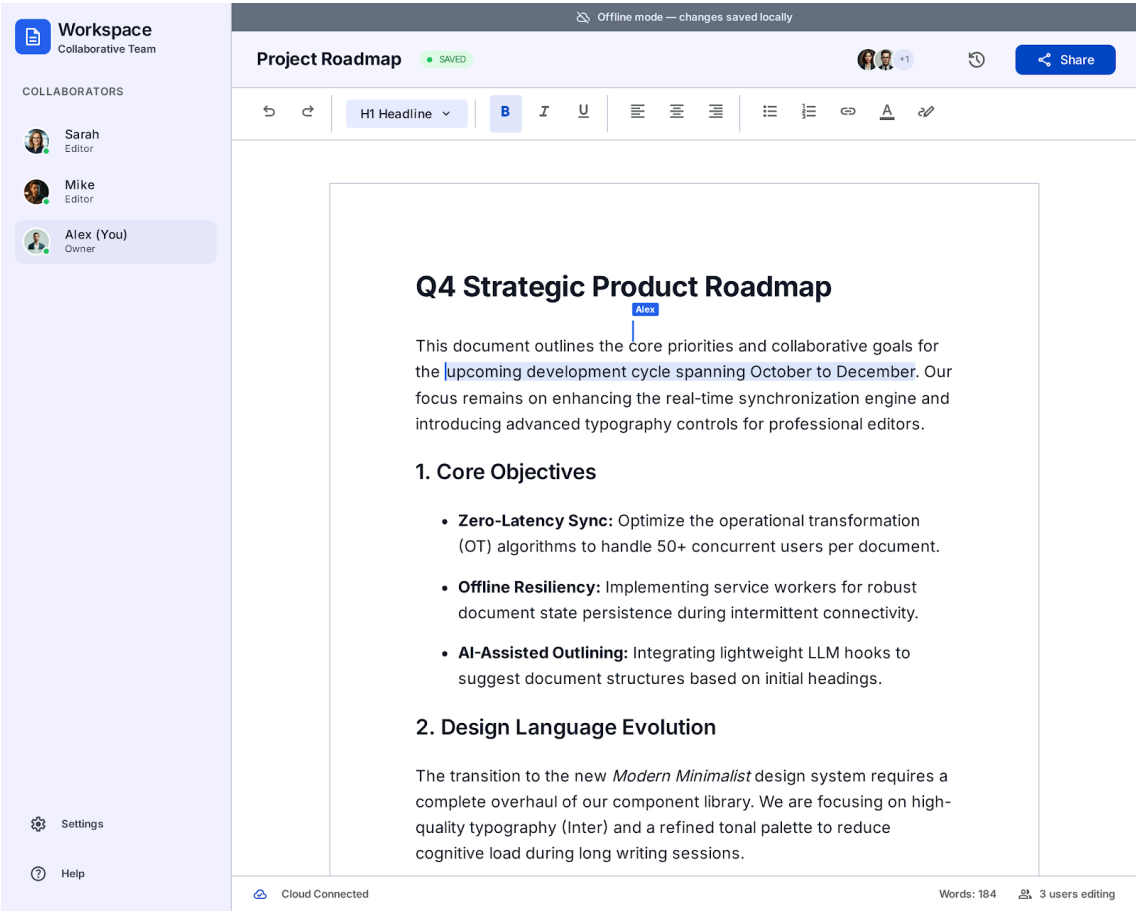
Hình 2.1. Giao diện đăng nhập/đăng ký

2.3.3. Trang danh sách tài liệu



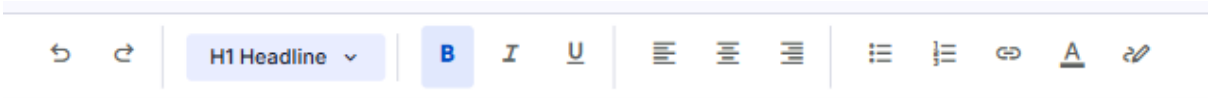
Hình 2.2. Giao diện trang danh sách tài liệu

2.3.4. *Trang soạn thảo tài liệu*



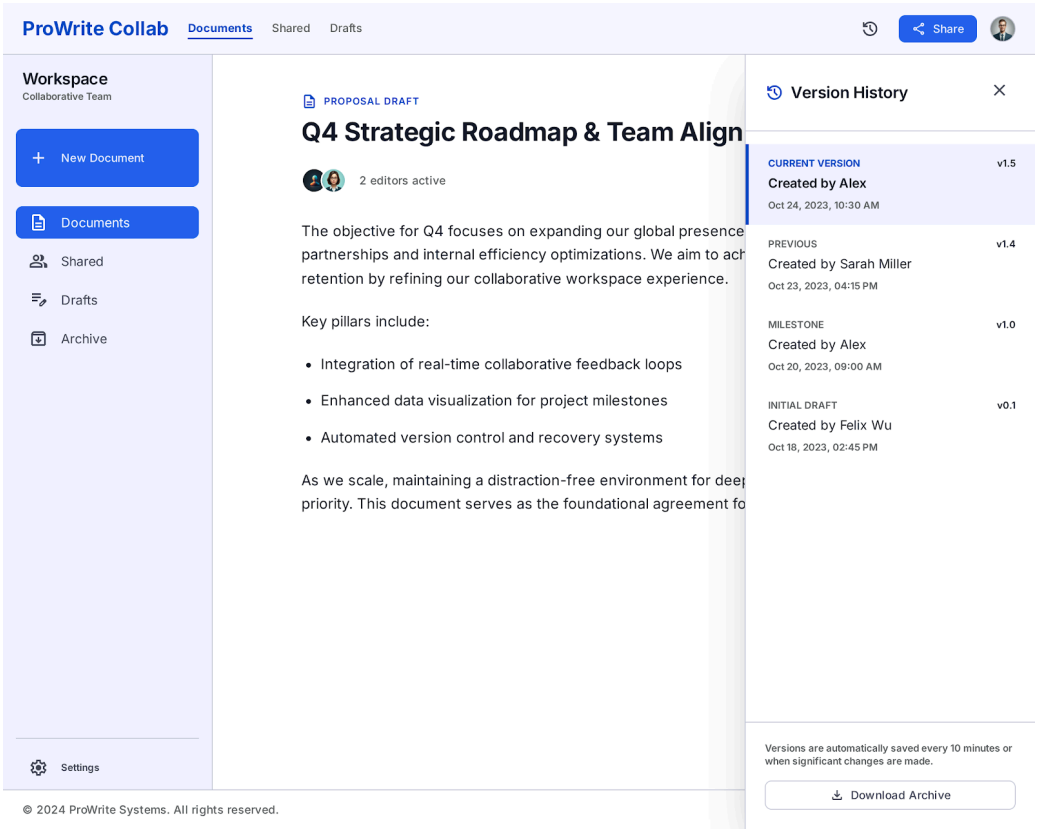
Hình 2.3. *Giao diện trang soạn thảo tài liệu*

2.3.5. *Toolbar rich text*



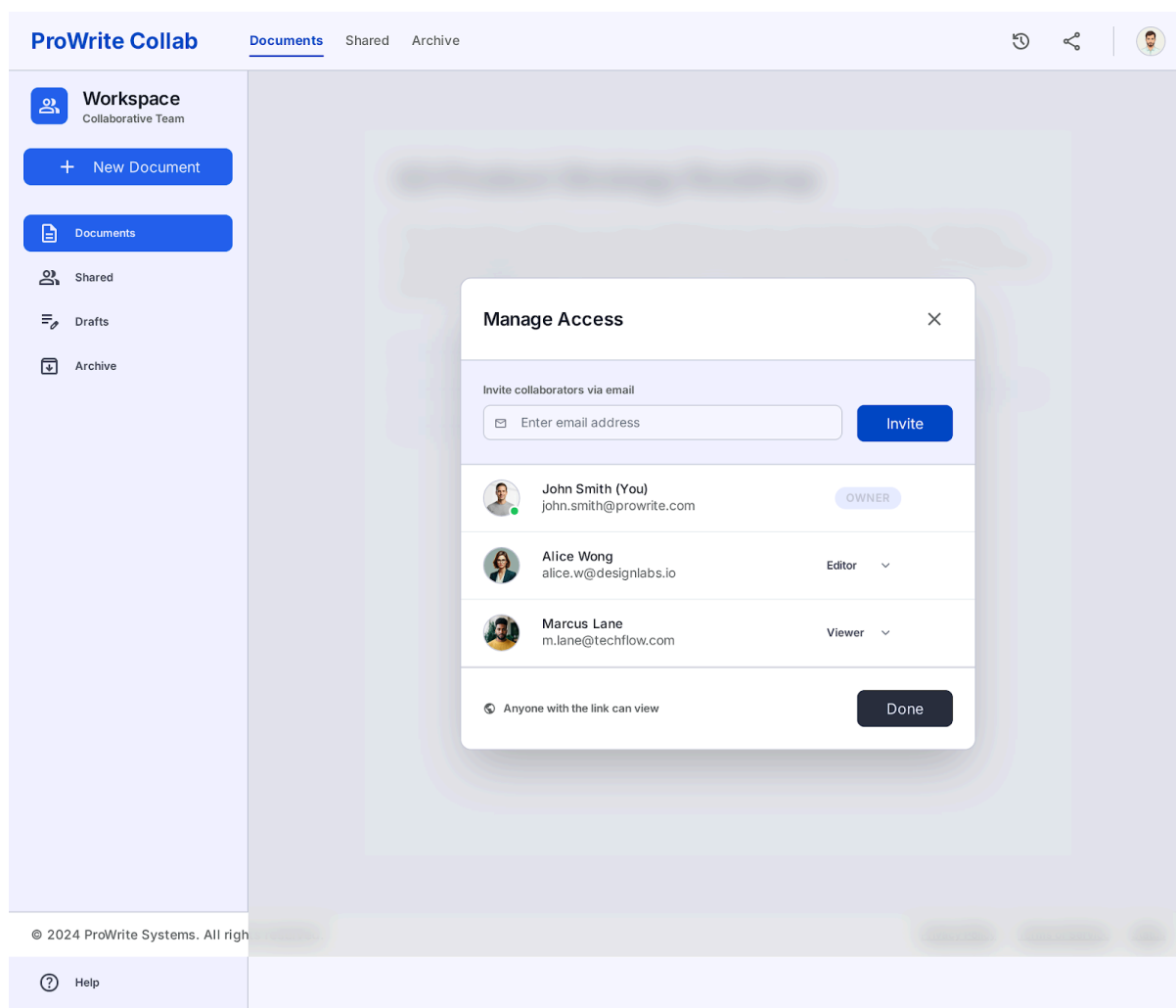
Hình 2.4. *Giao diện toolbar*

2.3.6. Phần lịch sử phiên bản



Hình 2.5. Giao diện phần lịch sử chỉnh sửa phiên bản

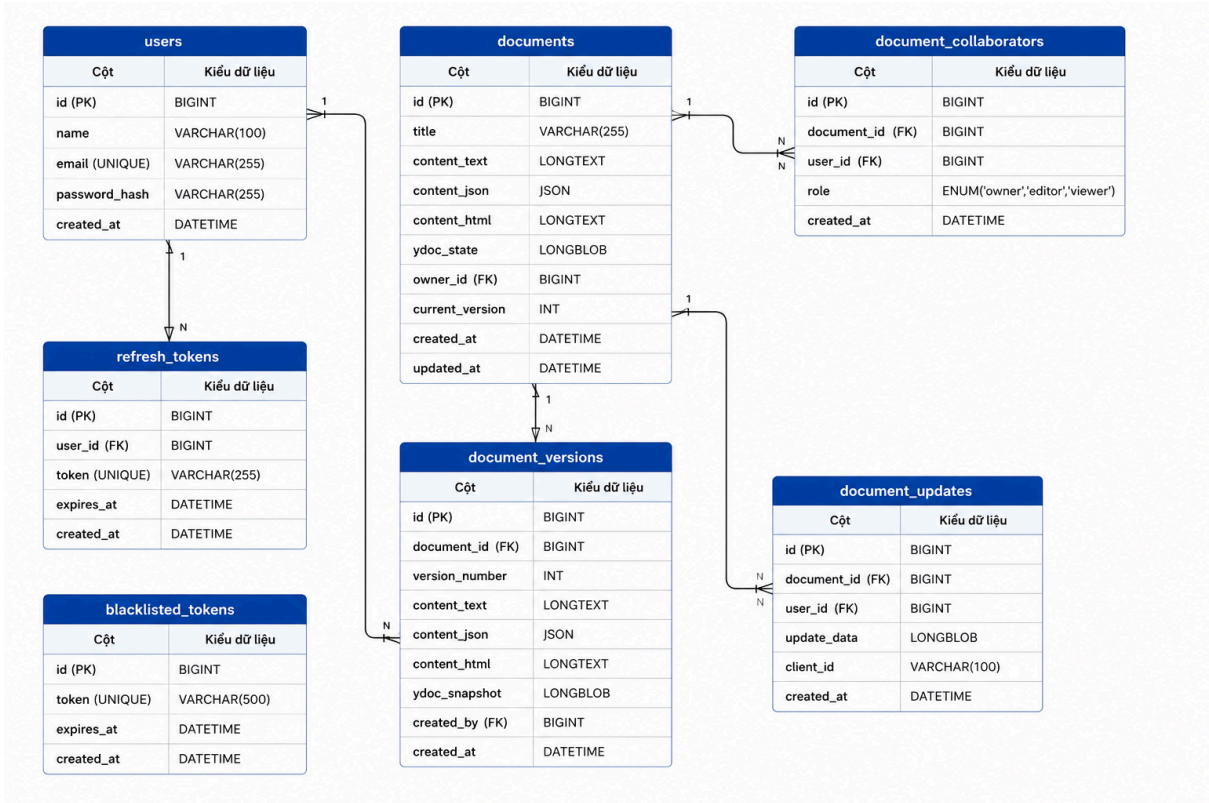
2.3.7. Phần thiết lập quyền cộng tác



Hình 2.6. Giao diện phần thiết lập quyền cộng tác

2.4. Thiết kế cơ sở dữ liệu

Hệ thống sử dụng hệ quản trị cơ sở dữ liệu quan hệ MySQL với tên schema `collab_docs` để lưu trữ toàn bộ dữ liệu nghiệp vụ. Phía backend tích hợp thư viện Sequelize ORM nhằm ánh xạ các bảng trong MySQL thành các lớp mô hình (model), giúp việc thao tác với dữ liệu trở nên an toàn, nhất quán và dễ bảo trì hơn so với viết câu truy vấn SQL thuần túy. Cấu trúc cơ sở dữ liệu được tổ chức thành sáu nhóm bảng chính phục vụ các chức năng: quản lý người dùng, lưu trữ tài liệu, phân quyền cộng tác, lịch sử phiên bản, đồng bộ thời gian thực và quản lý phiên đăng nhập.



Hình 2.7. Bảng cơ sở dữ liệu quan hệ

2.4.1. Bảng users

Bảng 2.2. Bảng users

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AI	ID người dùng
name	VARCHAR(100)	NN	Tên hiển thị
email	VARCHAR(255)	UQ	Email đăng nhập
password_hash	VARCHAR(255)		Mật khẩu (băm)
created_at	DATETIME	Def: NOW	Ngày tạo tài khoản

2.4.2. Bảng documents

Bảng 2.3. Bảng documents

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AI	ID tài liệu
title	VARCHAR(255)	NN	Tiêu đề
content_text	LONGTEXT		Text thuần (tìm kiếm)
content_json	JSON		Rich text JSON
content_html	LONGTEXT		Mã HTML (hiển thị)
ydoc_state	LOB		Trạng thái Yjs (nhị phân)
owner_id	BIGINT	NN, FK	ID chủ sở hữu
current_version	INT	Def: 1	Phiên bản hiện tại
created_at	DATETIME	Def: NOW	Ngày tạo
updated_at	DATETIME	Def: NOW	Ngày cập nhật gần nhất

2.4.3. Bảng document_collaborators

Bảng 2.4. Bảng document_collaborators

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AI	Khóa chính
document_id	BIGINT	NN, FK	ID tài liệu (Cascade)
user_id	BIGINT	NN, FK	ID người dùng (Cascade)
role	ENUM	NN, Def: 'editor'	Gồm: owner, editor, viewer
created_at	DATETIME	Def: NOW	Ngày cấp quyền

2.4.4. Bảng *document_versions*

Bảng 2.5. Bảng *document_versions*

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AI	Khóa chính
document_id	BIGINT	NN, FK	ID tài liệu (Cascade)
version_number	INT	NN	Số thứ tự phiên bản
content_text	LONGTEXT		Text thuần tại mốc lưu
content_json	JSON		Bản JSON tại mốc lưu
content_html	LONGTEXT		Mã HTML tại mốc lưu
ydoc_snapshot	LOB		Snapshot Yjs (nhị phân)
created_by	BIGINT	FK	ID người tạo bản lưu
created_at	DATETIME	Def: NOW	Thời gian lưu

2.4.5. Bảng *document_updates*

Bảng 2.6. Bảng *document_updates*

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AI	Khóa chính
document_id	BIGINT	NN, FK	ID tài liệu (Cascade)
user_id	BIGINT	FK	ID người gửi update
update_data	LOB	NN	Gói update Yjs (binary)
client_id	VARCHAR(100)		ID kết nối Socket
created_at	DATETIME	Def: NOW	Thời gian nhận

2.4.6. Bảng refresh_tokens và blacklisted_tokens

Bảng 2.7. Bảng refresh_tokens

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AI	Khóa chính
user_id	BIGINT	NN, FK	ID người dùng (Cascade)
token	VARCHAR(255)	NN, UQ	Mã Refresh Token
expires_at	DATETIME	NN	Hạn sử dụng
created_at	DATETIME	Def: NOW	Ngày tạo mã

Bảng 2.8. Bảng blacklisted_tokens

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AI	Khóa chính
token	VARCHAR(500)	NN, UQ	Access Token bị chặn
expires_at	DATETIME	NN	Hạn sử dụng (để dọn dẹp)
created_at	DATETIME	Def: NOW	Ngày đưa vào danh sách đen

2.5. Thiết kế lưu trữ nội dung tài liệu

Một trong những quyết định thiết kế quan trọng của hệ thống là không lưu nội dung tài liệu dưới một dạng duy nhất mà phân rã thành bốn trường riêng biệt, mỗi trường phục vụ một mục đích nghiệp vụ rõ ràng. Cách tiếp cận này xuất phát từ thực tế rằng các thao tác khác nhau trong hệ thống có yêu cầu về định dạng dữ liệu khác nhau: hiển thị danh sách tài liệu cần dữ liệu nhẹ và nhanh, render giao diện soạn thảo cần giữ đầy đủ cấu trúc định dạng, còn phục hồi phiên cộng tác cần trạng thái Yjs chính xác đến từng thao tác.

2.5.1. Lưu content_text

Trường content_text kiểu LONGTEXT lưu nội dung tài liệu dưới dạng văn bản thuần túy (plain text), được trích xuất từ editor thông qua lệnh editor.getText() của TipTap. Trường này loại bỏ toàn bộ thông tin định dạng như heading, bold hay màu chữ, chỉ giữ lại chuỗi ký tự thô.

Mục đích chính của content_text là phục vụ hai nghiệp vụ nhẹ tải: hiển thị đoạn preview ngắn trong danh sách tài liệu mà không cần parse JSON phức tạp, và làm nguồn dữ liệu cho tính năng tìm kiếm toàn văn (full-text search) trên MySQL. Do không mang thông tin định dạng, content_text không được dùng để phục hồi nội dung soạn thảo.

2.5.2. Lưu content_json

Trường content_json kiểu JSON lưu cấu trúc tài liệu của TipTap dưới dạng đối tượng ProseMirror document, được lấy qua lệnh editor.getJSON(). Đây là trường quan trọng nhất về mặt định dạng nội dung, vì nó mã hóa đầy đủ cây cấu trúc tài liệu bao gồm loại node (paragraph, heading, bulletList, ...), các mark định dạng (bold, italic, color, ...) và thuộc tính căn lề hay liên kết.

content_json đóng vai trò là nguồn chân lý (source of truth) cho rich text và được sử dụng trong hai trường hợp chính: tải nội dung tài liệu lần đầu khi chưa có ydoc_state hợp lệ thông qua lệnh editor.commands.setContent(document.contentJson), và phục hồi nội dung hiển thị khi xem lại lịch sử phiên bản. MySQL hỗ trợ kiểu JSON native từ phiên bản 5.7.8, cho phép truy vấn trực tiếp vào các trường con của JSON nếu cần.

2.5.3. Lưu content_html

Trường content_html kiểu LONGTEXT lưu nội dung tài liệu dưới dạng chuỗi HTML, được trích xuất qua lệnh editor.getHTML(). Chuỗi HTML này phản ánh đầy đủ định dạng hiển thị của tài liệu tại thời điểm lưu, bao gồm các thẻ <h1>, , , , <a> và các thuộc tính style inline tương ứng.

Mục đích của content_html là phục vụ các tình huống cần render nội dung nhanh ở phía server hoặc phía client mà không cần khởi tạo editor TipTap, chẳng hạn như tạo bản xem trước (preview) có định dạng, xuất nội dung ra file hoặc nhúng nội

dung tài liệu vào email thông báo. Đây là dạng dữ liệu thứ cấp, không được dùng để phục hồi trạng thái soạn thảo.

2.5.4. Lưu ydoc_state

Trường ydoc_state kiểu LONGBLOB lưu trạng thái nhị phân của Y.Doc — đối tượng tài liệu trung tâm của Yjs — được mã hóa thông qua hàm Y.encodeStateAsUpdate(ydoc) rồi chuyển sang Base64 trước khi truyền qua API. Khác với ba trường còn lại mang tính chất hiển thị, ydoc_state mang đầy đủ thông tin về vector clock và lịch sử thao tác của Yjs, đủ để tái tạo chính xác trạng thái cộng tác của tài liệu.

ydoc_state được sử dụng khi một client kết nối vào tài liệu đang có người khác chỉnh sửa: server trả về ydoc_state mới nhất, client khởi tạo Y.Doc từ trạng thái này rồi tiếp tục nhận các Yjs update incremental tiếp theo qua Socket.IO, đảm bảo hội tụ về cùng trạng thái với các client khác mà không cần replay toàn bộ lịch sử update từ đầu. Trường này cũng được sao lưu vào bảng document_versions dưới tên ydoc_snapshot để phục vụ khôi phục phiên cộng tác về một mốc lịch sử cụ thể.

2.6. Thiết kế cơ chế đồng bộ realtime

2.6.1. Cơ chế Yjs update

Cơ chế cập nhật dữ liệu của Yjs là nền tảng cốt lõi giúp hệ thống thực hiện đồng bộ hóa nội dung thời gian thực giữa nhiều người dùng mà không gây ra xung đột. Thay vì truyền tải toàn bộ cấu trúc tài liệu sau mỗi thao tác chỉnh sửa, hệ thống sử dụng thuật toán CRDT (Conflict-free Replicated Data Type) để phân rã các thay đổi thành những gói dữ liệu cập nhật nhị phân (update binary) siêu nhỏ. Những gói dữ liệu này đại diện cho sự thay đổi nhỏ nhất của văn bản, giúp tối ưu hóa đáng kể băng thông mạng và giảm thiểu độ trễ cho người dùng trong quá trình cộng tác.

Khi một người dùng thực hiện chỉnh sửa trên giao diện TipTap, Yjs lập tức ghi nhận thao tác đó và tạo ra các bản cập nhật tương ứng dựa trên trạng thái của Y.Doc. Các gói cập nhật này sau đó được truyền tải tới máy chủ thông qua kết nối WebSocket của Socket.IO. Việc truyền tải các update nhỏ thay vì toàn bộ nội dung tài liệu là giải pháp chiến lược giúp hệ thống vận hành mượt mà ngay cả khi có nhiều người cùng

chỉnh sửa đồng thời, tránh tình trạng quá tải dữ liệu gây gián đoạn trải nghiệm người dùng.

Tại phía máy chủ, sau khi tiếp nhận các gói cập nhật từ client, tầng service sẽ thực hiện quy trình kiểm tra quyền hạn nghiêm ngặt để đảm bảo người gửi có vai trò chủ sở hữu hoặc biên tập viên hợp lệ. Sau khi vượt qua vòng kiểm duyệt, máy chủ tiến hành ghi lại thay đổi vào bảng `document_updates` trong cơ sở dữ liệu và thực hiện gộp (merge) dữ liệu vào trạng thái tài liệu gốc (`ydoc_state`). Cuối cùng, máy chủ phát quảng bá (broadcast) chính xác gói cập nhật đó tới tất cả các kết nối client khác trong cùng phòng ảo, đảm bảo mọi thành viên đều đạt đến tính nhất quán cuối cùng của tài liệu.

2.6.2. Cơ chế *join document room*

Quy trình gia nhập phòng tài liệu là bước khởi đầu thiết yếu để thiết lập phiên làm việc cộng tác giữa ứng dụng khách và máy chủ. Khi người dùng mở một tài liệu, ứng dụng client sẽ chủ động phát ra sự kiện `join-document` thông qua kết nối Socket.IO, kèm theo các thông tin định danh quan trọng bao gồm mã số tài liệu (`documentId`) và thông tin cá nhân của người dùng. Việc này giúp máy chủ nhận diện được yêu cầu kết nối và bắt đầu quy trình xác thực phiên làm việc trong thời gian thực.

Sau khi tiếp nhận sự kiện, máy chủ thực hiện một bước kiểm tra quyền truy cập nghiêm ngặt đối với tài liệu được yêu cầu dựa trên dữ liệu người dùng được lưu trữ trong bảng `document_collaborators`. Chỉ khi người dùng sở hữu vai trò hợp lệ (`owner`, `editor`, hoặc `viewer`), máy chủ mới chấp nhận cho phép client tham gia vào phiên làm việc. Cơ chế kiểm tra này đảm bảo rằng không một cá nhân nào có thể truy cập vào tài liệu nếu không được cấp phép, từ đó bảo vệ tối đa tính an toàn thông tin.

Khi các điều kiện xác thực được thỏa mãn, máy chủ tiến hành gán kết nối của client đó vào một không gian phòng ảo độc lập, được định danh duy nhất theo mã tài liệu trên hệ thống Socket.IO. Việc cô lập kết nối vào từng phòng riêng biệt giúp máy chủ tối ưu hóa hiệu suất xử lý, vì mọi thông tin cập nhật sau đó chỉ cần được phát quảng bá tới các thành viên nằm trong phạm vi phòng ảo này, thay vì toàn bộ hệ thống.

Cuối cùng, sau khi tham gia phòng thành công, máy chủ sẽ ghi nhớ trạng thái kết nối này vào bộ nhớ đệm để tăng tốc độ phản hồi cho các thao tác tiếp theo. Đồng thời, một thông báo về danh sách các thành viên hiện đang trực tuyến sẽ được gửi tới tất cả người dùng trong phòng, giúp các cộng tác viên nắm bắt được sự hiện diện của nhau. Quy trình này khép lại bằng việc thiết lập sự sẵn sàng cho luồng dữ liệu cộng tác, cho phép người dùng bắt đầu các thao tác chỉnh sửa văn bản ngay lập tức.

2.6.3. Cơ chế *user online* và *cursor*

Để nâng cao trải nghiệm cộng tác, hệ thống triển khai cơ chế nhận thức không gian (awareness) thông qua module Awareness của Yjs, giúp người dùng nắm bắt được sự hiện diện và hoạt động của các cộng tác viên. Ngay khi client gia nhập vào phòng tài liệu, hệ thống tự động khởi tạo trạng thái trực tuyến của người dùng. Trạng thái này bao gồm các thông tin định danh như tên người dùng, màu sắc đại diện cho con trỏ (cursor) và các thông số tùy chỉnh khác, giúp việc phân biệt người tham gia trở nên trực quan và dễ dàng hơn trong quá trình soạn thảo.

Khi người dùng di chuyển chuột hoặc thực hiện thao tác chọn văn bản trên trình soạn thảo TipTap, client sẽ gửi các sự kiện `cursor-update` và `selection-update` tới máy chủ thông qua giao thức WebSocket. Các sự kiện này mang theo dữ liệu về tọa độ con trỏ hoặc dải văn bản đang được bôi đen. Thay vì lưu trữ bền vững vào cơ sở dữ liệu, các gói dữ liệu này được máy chủ xử lý như các sự kiện mạng tạm thời (transient events). Máy chủ có trách nhiệm chuyển tiếp các gói tin này đến tất cả các client khác đang hiện diện trong cùng phòng ảo để cập nhật vị trí con trỏ của cộng tác viên tương ứng.

Việc thiết kế cơ chế này như một luồng dữ liệu tạm thời mang lại lợi ích lớn về mặt tối ưu hóa tài nguyên hệ thống. Bằng cách không ghi lại các thao tác di chuyển chuột vào cơ sở dữ liệu, máy chủ tránh được tình trạng quá tải do hàng ngàn sự kiện nhỏ lẻ gây ra, đồng thời vẫn đảm bảo tính tức thời cho trải nghiệm người dùng. Nhờ vào sự truyền tải liên tục của dữ liệu nhận thức, mỗi thành viên đều có thể quan sát thấy vị trí làm việc của cộng tác viên khác, tạo ra sự tương tác đồng bộ và cảm giác như đang cùng làm việc trực tiếp trong một không gian thực.

Cuối cùng, cơ chế này còn tự động quản lý việc ngắt kết nối. Khi một cộng tác viên rời khỏi phòng hoặc mất kết nối mạng đột ngột, máy chủ sẽ phát hiện sự gián đoạn của luồng WebSocket và lập tức thông báo tới các thành viên còn lại để xóa bỏ con trỏ của người đó khỏi giao diện. Điều này giúp hệ thống luôn giữ được sự gọn gàng và chính xác về danh sách thành viên hiện hữu, đảm bảo rằng mọi thông tin về người đang trực tuyến luôn là thông tin mới nhất và xác thực nhất trong suốt phiên làm việc nhóm.

2.6.4. Cơ chế *offline sync*

Khả năng làm việc ngoại tuyến (offline editing) là một yêu cầu kỹ thuật quan trọng giúp hệ thống đảm bảo trải nghiệm liền mạch ngay cả khi người dùng đối mặt với kết nối mạng không ổn định hoặc bị gián đoạn đột ngột. Để hiện thực hóa điều này, hệ thống tận dụng cơ chế lưu trữ cục bộ của thư viện Yjs kết hợp với IndexedDB ngay trên trình duyệt của người dùng. Trong thời gian ngoại tuyến, mọi thao tác chỉnh sửa văn bản không bị mất đi mà được Yjs tự động ghi lại dưới dạng các gói cập nhật nhị phân (update) và lưu trữ trực tiếp vào IndexedDB, đóng vai trò như một bản ghi tạm thời an toàn cho đến khi kết nối được thiết lập lại.

Ngay khi thiết bị của người dùng khôi phục được kết nối với máy chủ thông qua Socket.IO, tiến trình đồng bộ hóa dữ liệu (sync) sẽ tự động được kích hoạt. Client sẽ gửi toàn bộ các gói cập nhật còn tồn đọng (pending updates) đã được lưu trữ trong IndexedDB lên máy chủ. Tại phía máy chủ, quy trình hợp nhất (merge) dữ liệu sẽ diễn ra dựa trên thuật toán CRDT, đảm bảo các thay đổi được thực hiện khi ngoại tuyến sẽ được tích hợp một cách chính xác vào cấu trúc dữ liệu chung của tài liệu mà không làm xung đột hoặc xóa bỏ các thay đổi đã được thực hiện bởi các cộng tác viên khác trong thời gian người dùng bị gián đoạn kết nối.

Để duy trì tính nhất quán sau quá trình đồng bộ, hệ thống sử dụng các sự kiện mạng đặc thù như reconnect-sync và sync-completed nhằm quản lý luồng dữ liệu giữa client và server. Các sự kiện này cho phép cả hai phía kiểm tra trạng thái vector đồng bộ (state vector), từ đó máy chủ có thể gửi trả các đoạn dữ liệu thiếu hụt để đưa client về trạng thái tài liệu mới nhất. Quá trình này được thực hiện hoàn toàn tự động ở chế độ nền, giúp người dùng không cần phải thực hiện bất kỳ thao tác thủ công nào để

khôi phục lại dữ liệu, đồng thời củng cố niềm tin về tính sẵn sàng và độ tin cậy của hệ thống trong mọi điều kiện vận hành.

2.6.5. Cơ chế undo/redo trong cộng tác

Cơ chế hoàn tác (Undo) và làm lại (Redo) trong môi trường cộng tác thời gian thực đòi hỏi sự kiểm soát tinh tế để tránh việc can thiệp chéo vào công việc của người khác. Hệ thống triển khai Y.UndoManager với phạm vi quản lý được định nghĩa theo từng người dùng (user-scoped). Điều này có nghĩa là mỗi khi một thành viên thực hiện thao tác Undo, trình quản lý chỉ nhận diện và thu hồi các thay đổi do chính người đó khởi tạo trong phiên làm việc hiện tại, thay vì khôi phục toàn bộ trạng thái tài liệu của tất cả cộng tác viên. Việc giới hạn phạm vi này là yếu tố then chốt giúp duy trì sự ổn định, đảm bảo hành động của một người không vô tình làm mất đi nội dung giá trị do các thành viên khác vừa đóng góp.

Khi một người dùng kích hoạt lệnh Undo, Y.UndoManager sẽ trích xuất các thao tác gần nhất trong lịch sử hành động cục bộ và đảo ngược chúng thông qua việc tạo ra các gói cập nhật Yjs tương ứng. Những bản cập nhật "đảo ngược" này ngay lập tức được truyền tải qua Socket.IO tới máy chủ, sau đó được phát quảng bá tới tất cả các client khác đang cùng làm việc trong phòng ảo. Khi các client nhận được gói cập nhật này, thuật toán CRDT của Yjs sẽ thực thi việc hợp nhất, giúp nội dung tài liệu trên mọi thiết bị được cập nhật đồng bộ theo kết quả của thao tác hoàn tác vừa thực hiện.

Để đảm bảo tính nhất quán xuyên suốt, hệ thống phân tách rõ ràng giữa các hành động mang tính chất thay đổi nội dung (như gõ văn bản, định dạng rich text) và các hành động điều hướng (như di chuyển con trỏ). Chỉ những thao tác làm thay đổi nội dung tài liệu mới được đưa vào hàng đợi của UndoManager. Điều này ngăn chặn việc người dùng vô tình hoàn tác các di chuyển chuột hoặc các thay đổi tạm thời, giúp luồng làm việc luôn tập trung vào giá trị nội dung cốt lõi. Nhờ vào việc kết hợp chặt chẽ giữa phạm vi quản lý cá nhân và thuật toán hợp nhất CRDT, hệ thống đạt được sự cân bằng tối ưu giữa tính linh hoạt của người dùng và tính an toàn dữ liệu trong cộng tác nhóm.

CHƯƠNG 3. KẾT QUẢ CÀI ĐẶT VÀ ĐÁNH GIÁ

3.1. Môi trường cài đặt

Để triển khai hệ thống soạn thảo cộng tác thời gian thực ổn định, môi trường cài đặt cần đáp ứng các tiêu chuẩn kỹ thuật về cả phần cứng và phần mềm, phục vụ việc xử lý đồng thời nhiều kết nối Socket.IO và hợp nhất cấu trúc dữ liệu CRDT Yjs trên bộ nhớ.

Về phần cứng, máy chủ dịch vụ (Backend & Database Node) yêu cầu bộ vi xử lý tối thiểu Intel Core i5/i7 thế hệ 10 hoặc CPU Xeon (4 nhân, 8 luồng, xung nhịp từ 2.5 GHz trở lên) để xử lý giải mã nhị phân Yjs nhanh chóng, hạn chế tối đa độ trễ. Bộ nhớ trong máy chủ yêu cầu tối thiểu 8 GB RAM (khuyến dùng 16 GB) làm bộ đệm cho đối tượng Y.Doc hoạt động trực tiếp trong Node.js và bộ đệm MySQL Buffer Pool. Thiết bị lưu trữ bắt buộc sử dụng ổ cứng SSD (SATA III hoặc NVMe) trống tối thiểu 10 GB để đáp ứng tốc độ đọc/ghi ngẫu nhiên (Random IOPS) cao khi liên tục lưu trữ các gói thay đổi Yjs (document_updates) xuống CSDL. Băng thông mạng yêu cầu tối thiểu 100 Mbps ổn định. Phía máy trạm của người dùng (Client Node) chỉ cần cấu hình văn phòng cơ bản với CPU Intel Core i3 và RAM từ 4 GB đến 8 GB để vận hành trình duyệt mượt mà.

Về phần mềm, hệ thống hoạt động đa nền tảng từ môi trường phát triển Windows 10/11 64-bit đến máy chủ thực tế chạy Linux Ubuntu Server (20.04/22.04 LTS). Máy chủ dịch vụ vận hành trên môi trường Node.js v20.x/v22.x (LTS) cùng trình quản lý gói npm v10.x, tận dụng kiến trúc bất đồng bộ (Asynchronous) và phi chặn (Non-blocking I/O) của Node.js để duy trì hàng ngàn kết nối WebSocket đồng thời qua Socket.IO. Hệ quản trị cơ sở dữ liệu sử dụng MySQL Server v8.0 hoặc v5.7 nhờ khả năng lưu trữ trực tiếp cấu trúc cây Rich Text của TipTap ở định dạng JSON bản địa và lưu trữ trạng thái nhị phân nén Yjs (ydoc_state và update_data) nguyên bản bằng kiểu dữ liệu LONGBLOB (lên đến 4 GB). Phía máy trạm người dùng sử dụng các trình duyệt hiện đại (Chrome v110+, Edge v110+, Firefox v108+) hỗ trợ HTML5 WebSockets để truyền tin hai chiều, ES Modules để nạp thư viện Yjs qua CDN esm.sh và bộ nhớ IndexedDB để lưu đệm soạn thảo ngoại tuyến (Offline Mode).

3.2. Cấu hình và chạy hệ thống

Trước khi cài đặt hệ thống, tạo file `.env` và copy nội dung từ file `.env.example` vào. Sau đó thiết lập lại biến tùy theo môi trường cài đặt là Docker hoặc dùng Local.

3.2.1. Cài đặt với Docker

- **Chạy lần đầu:** Đứng ở thư mục gốc, mở terminal và chạy câu lệnh:

```
docker compose up -d --build
```

- **Chạy lần sau:**

```
docker compose up -d
```

- **Chạy lại với dữ liệu mới:**

```
docker compose down -v
```

```
docker compose up -d --build
```

3.2.2. Cài đặt trên Local (Không dùng Docker)

- **Cài đặt thư viện:**

```
npm install:all
```

- **Chạy ứng dụng:**

```
npm run dev
```

Lưu ý: Với trường hợp chạy trên local không dùng docker, phải đổi lại thiết lập `.env` cho phù hợp với mysql của máy local.

3.3. Kết quả đạt được

Sau quá trình nghiên cứu, phân tích thiết kế và tiến hành lập trình cài đặt, nhóm đã xây dựng thành công "Hệ thống soạn thảo văn bản cộng tác thời gian thực", đáp ứng tốt các mục tiêu ban đầu đề ra ở cả hai phương diện: chức năng nghiệp vụ và yêu cầu kỹ thuật phân tán. Cụ thể, những kết quả nổi bật hệ thống đã đạt được bao gồm:

3.3.1. Về mặt chức năng nghiệp vụ

- **Hoàn thiện luồng xác thực và bảo mật:** Xây dựng thành công cơ chế quản lý định danh người dùng sử dụng chuỗi mã thông báo JWT (Access Token và Refresh Token) an toàn. Triển khai trọn vẹn quy trình đăng ký, đăng nhập và đăng xuất (có sử dụng cơ chế Blacklist Token để vô hiệu hóa phiên làm việc ngay lập tức).

- **Quản lý không gian làm việc cá nhân hóa:** Cung cấp cho người dùng giao diện bảng điều khiển (Dashboard) trực quan để khởi tạo, xem trước (preview), đổi tên, xóa và nhân bản (copy) tài liệu một cách linh hoạt. Tích hợp tính năng tự động lưu (autosave) để ngăn ngừa rủi ro mất mát dữ liệu.
- **Soạn thảo văn bản giàu định dạng (Rich text):** Tích hợp thành công bộ công cụ TipTap, cung cấp trải nghiệm soạn thảo chuyên nghiệp với đầy đủ các định dạng như: in đậm, in nghiêng, gạch chân, tạo tiêu đề, danh sách, căn lề, chèn liên kết, đổi màu chữ và tô sáng nền văn bản.
- **Cộng tác thời gian thực (Realtime Collaboration):** Đây là kết quả cốt lõi của đề tài. Hệ thống cho phép nhiều người dùng cùng lúc chỉnh sửa trên một tài liệu mà nội dung vẫn được đồng bộ mượt mà, độ trễ thấp và không xảy ra tình trạng mất mát hay ghi đè dữ liệu lên nhau.
- **Nhận thức không gian làm việc (Awareness):** Hiển thị chính xác danh sách các thành viên đang trực tuyến trong phòng, đồng thời trực quan hóa vị trí con trỏ chuột (cursor) và vùng văn bản đang bôi đen của từng người dùng với các màu sắc đại diện riêng biệt.
- **Phân quyền và bảo mật tài liệu:** Triển khai chặt chẽ cơ chế phân quyền ba cấp độ (Chủ sở hữu - Biên tập viên - Người xem). Các đặc quyền này được hệ thống Backend kiểm duyệt gắt gao ở cả luồng HTTP API và sự kiện WebSocket.
- **Quản lý lịch sử phiên bản:** Xây dựng thành công tính năng lưu trữ mốc lịch sử (snapshot) chứa đầy đủ cấu trúc định dạng. Cho phép người dùng xem lại nội dung quá khứ và tiến hành khôi phục (restore) toàn vẹn khi cần thiết.

3.3.2. Về mặt kỹ thuật và kiến trúc phân tán

- **Ứng dụng thành công cấu trúc dữ liệu CRDT:** Hệ thống đã tích hợp và vận hành trơn tru thư viện Yjs, hiện thực hóa cơ chế hội tụ nhất quán cuối cùng (Eventual Consistency) đặc trưng của các hệ thống phân tán. Thuật toán này giúp giải quyết xung đột dữ liệu tự động ngay tại các máy trạm (client) mà không cần máy chủ phải tốn tài nguyên phân xử phức tạp.

- **Thiết kế lưu trữ đa định dạng (Multi-representation):** Xây dựng thành công mô hình cơ sở dữ liệu MySQL tối ưu hóa bằng cách phân tách nội dung tài liệu thành bốn biểu diễn: văn bản thuần (phục vụ tìm kiếm), mã JSON (bảo toàn cấu trúc gốc), mã HTML (kết xuất nhanh) và trạng thái nhị phân Yjs (phục hồi tiến trình cộng tác).
- **Tích hợp năng lực đồng bộ ngoại tuyến (Offline Sync):** Vận dụng IndexedDB tại trình duyệt để cho phép người dùng tiếp tục thao tác soạn thảo khi rớt mạng, sau đó hệ thống tự động hòa trộn (merge) các gói cập nhật bị trễ bằng vector trạng thái (State Vector) ngay khi kết nối Internet được khôi phục.
- **Kiến trúc Client-Server hiện đại:** Tổ chức mã nguồn linh hoạt, phân định rõ ràng lớp giao diện (React.js) và lớp nghiệp vụ (Node.js/Express). Việc liên lạc được thiết kế song song qua RESTful API phục vụ luồng chuẩn và Socket.IO phục vụ luồng thời gian thực.

Nhìn chung, sản phẩm của đề tài là một minh chứng thực tiễn, khẳng định khả năng vận dụng thành thạo các kiến thức lý thuyết về kiến trúc phân tán, giao thức truyền tin và cấu trúc dữ liệu CRDT vào việc giải quyết một bài toán phần mềm thực tế có độ phức tạp cao.

3.4. Hạn chế của hệ thống

Mặc dù hệ thống đã xây dựng được nền tảng hỗ trợ soạn thảo văn bản cộng tác thời gian thực với cơ chế đồng bộ bằng Yjs và Socket.IO, hệ thống vẫn còn tồn tại một số hạn chế cần tiếp tục cải thiện.

Thứ nhất, hệ thống hiện mới tập trung vào các chức năng cộng tác cơ bản như chỉnh sửa đồng thời, hiển thị con trỏ người dùng, lưu lịch sử phiên bản và phân quyền truy cập. Các tính năng nâng cao thường xuất hiện trong các nền tảng cộng tác chuyên nghiệp như bình luận theo đoạn văn (comment threading), đề xuất chỉnh sửa (suggestion mode), theo dõi thay đổi (track changes), thông báo theo thời gian thực hoặc nhắc người dùng chưa được hỗ trợ.

Thứ hai, cơ chế lưu trữ hiện tại sử dụng kết hợp snapshot đầy đủ và các Yjs update nhỏ nhằm giảm chi phí đồng bộ. Tuy nhiên, khi tài liệu có kích thước lớn hoặc thời gian chỉnh sửa kéo dài, số lượng update tích lũy có thể tăng đáng kể, làm tăng chi

phí lưu trữ và thời gian xử lý khi phục hồi trạng thái hệ thống. Việc tối ưu chiến lược snapshot hoặc cơ chế nén dữ liệu hiện vẫn chưa được triển khai đầy đủ.

Thứ ba, hệ thống offline editing mới dừng ở mức hỗ trợ lưu trạng thái cục bộ và đồng bộ lại khi kết nối được khôi phục. Trong các trường hợp người dùng chỉnh sửa trong thời gian dài ở trạng thái ngoại tuyến hoặc xuất hiện nhiều thay đổi phức tạp đồng thời, hiệu năng đồng bộ và trải nghiệm người dùng có thể bị ảnh hưởng.

Thứ tư, cơ chế realtime hiện sử dụng Socket.IO trên một máy chủ tập trung. Kiến trúc này phù hợp với môi trường thử nghiệm hoặc hệ thống quy mô nhỏ đến trung bình, tuy nhiên khi số lượng người dùng và tài liệu hoạt động đồng thời tăng cao, hệ thống có thể gặp giới hạn về khả năng mở rộng. Hệ thống hiện chưa triển khai các giải pháp mở rộng như Redis Pub/Sub, Load Balancing hoặc phân tán WebSocket server.

Thứ năm, phân phân quyền hiện được thiết kế theo ba vai trò cơ bản gồm owner, editor và viewer. Mô hình này phù hợp với phạm vi đề tài nhưng chưa hỗ trợ các cơ chế phân quyền chi tiết hơn như quyền theo nhóm người dùng, quyền theo từng phần nội dung hoặc quyền truy cập có thời hạn.

Cuối cùng, hệ thống hiện chủ yếu tập trung vào chức năng và khả năng đồng bộ dữ liệu, chưa thực hiện đánh giá hiệu năng chuyên sâu trong các điều kiện tải lớn, ví dụ như số lượng lớn người dùng chỉnh sửa đồng thời hoặc tài liệu có nội dung rất dài. Do đó, khả năng hoạt động trong môi trường thực tế quy mô lớn cần tiếp tục được kiểm chứng và tối ưu trong các nghiên cứu tiếp theo.

3.5. Hướng phát triển

Hướng phát triển quan trọng đầu tiên của đề tài là việc tích hợp các mô hình Trí tuệ Nhân tạo (AI) vào quy trình soạn thảo nhằm biến hệ thống từ một công cụ soạn thảo thụ động thành một môi trường làm việc thông minh. Thay vì chỉ hỗ trợ các thao tác cơ bản của con người, hệ thống sẽ tích hợp một trợ lý AI hoạt động như một cộng tác viên thời gian thực. Trợ lý AI này sở hữu một thực thể kết nối riêng biệt qua giao thức Socket.IO, đồng thời đồng bộ hóa trực tiếp với cấu trúc dữ liệu nhị phân của Yjs. Khi nhận được yêu cầu từ người dùng thông qua các câu lệnh đặc tả hoặc thao tác bôi đen chọn văn bản, hệ thống sẽ chuyển đổi nội dung cấu trúc định dạng JSON của

TipTap gửi đến các mô hình ngôn ngữ lớn để phân tích. Câu trả lời được sinh ra từ AI sẽ được gửi ngược lại phòng làm việc dưới dạng các gói cập nhật nhị phân nhỏ, tạo ra hiệu ứng gõ chữ mượt mà và trực quan đối với tất cả người dùng mà không gây ra bất kỳ xung đột dữ liệu nào. Song song với đó, hệ thống sẽ áp dụng các mô hình nhúng văn bản để chuyển đổi nội dung tài liệu thành dạng biểu diễn vector đa chiều, lưu trữ vào cơ sở dữ liệu chuyên dụng để phục vụ tính năng tìm kiếm ngữ nghĩa sâu và gợi ý hoàn thành văn bản thông minh dưới dạng chữ mờ dựa trên ngữ cảnh soạn thảo tức thời.

Hướng phát triển tiếp theo tập trung vào việc nâng cấp hạ tầng phân tán nhằm cải thiện hiệu năng và khả năng chịu tải của hệ thống khi số lượng người dùng đồng thời tăng cao. Hiện tại, mô hình máy chủ đơn lẻ và cơ sở dữ liệu MySQL truyền thống có thể tạo ra các nút thắt cổ chai lớn khi phải xử lý liên tục các luồng dữ liệu thời gian thực. Do đó, hệ thống sẽ được chuyển đổi sang kiến trúc đa máy chủ đặt phía sau một bộ cân bằng tải. Để đảm bảo thông tin đồng bộ trạng thái Yjs và các sự kiện hiện diện người dùng được phát sóng đồng nhất xuyên suốt các máy chủ Backend độc lập, giải pháp sử dụng Redis làm kênh truyền tin trung gian sẽ được triển khai. Về phía tầng lưu trữ, thay vì ghi trực tiếp trạng thái nhị phân của tài liệu vào cơ sở dữ liệu MySQL sau mỗi thay đổi nhỏ, hệ thống sẽ sử dụng cơ chế lưu trữ đệm phân tán để tối ưu hóa tốc độ truy xuất dữ liệu tức thời. Đồng thời, cơ sở dữ liệu MySQL sẽ được tiến hành phân vùng dựa trên mã nhận diện tài liệu đối với các bảng lưu trữ bản ghi cập nhật gia tăng, giúp giảm thiểu đáng kể độ trễ truy vấn dữ liệu lịch sử và tăng cường tính sẵn sàng cao của toàn bộ hệ thống.

Cuối cùng, hệ thống hướng tới việc giảm thiểu sự phụ thuộc vào máy chủ trung tâm bằng cách tối ưu hóa cơ chế đồng bộ ngoại tuyến và mở rộng sang mô hình mạng ngang hàng. Nhằm giải quyết triệt để sự cố mất kết nối Internet toàn cục, hệ thống sẽ tích hợp giao thức WebRTC cho phép các thiết bị người dùng trong cùng một mạng nội bộ có thể trực tiếp tìm kiếm, kết nối và đồng bộ hóa tài liệu với nhau thông qua mạng ngang hàng mà không cần trung chuyển qua máy chủ. Khi mạng Internet được khôi phục, các thiết bị này sẽ tự động bầu chọn một Client đại diện có trạng thái dữ liệu cập nhật nhất để gửi các bản cập nhật tích lũy lên cơ sở dữ liệu trung tâm. Quá trình này sẽ sử dụng cơ chế so sánh vector trạng thái của Yjs để hòa trộn các nhánh dữ

liệu phân tán một cách chính xác, đưa toàn bộ hệ thống về trạng thái hội tụ nhất quán cuối cùng mà vẫn bảo toàn toàn bộ định dạng phong phú và lịch sử chỉnh sửa của tài liệu.

3.6. Kết luận chung

Trải qua quá trình nghiên cứu lý thuyết, thiết kế kiến trúc và triển khai thực nghiệm, đề tài đã xây dựng thành công một hệ thống soạn thảo văn bản cộng tác thời gian thực hoàn chỉnh, đáp ứng tốt các yêu cầu khắt khe của một hệ thống phân tán hiện đại. Bằng việc kết hợp hài hòa giữa các công nghệ tiên tiến bao gồm React, bộ soạn thảo rich text TipTap, thư viện đồng bộ Yjs sử dụng cấu trúc dữ liệu giải quyết xung đột CRDT, cùng kết nối thời gian thực Socket.IO và cơ sở dữ liệu MySQL, hệ thống đã giải quyết triệt để bài toán đồng bộ hóa trạng thái tài liệu khi có nhiều người dùng cùng tương tác đồng thời. Thành công lớn nhất của đề tài là việc hiện thực hóa cơ chế hội tụ nhất quán cuối cùng (Eventual Consistency), cho phép tích hợp mượt mà các thao tác định dạng văn bản phức tạp như tiêu đề, căn lề, danh sách, liên kết hay màu sắc mà hoàn toàn không xảy ra hiện tượng mất mát dữ liệu hoặc chồng chéo nội dung giữa các phiên làm việc độc lập.

Bên cạnh đó, giải pháp thiết kế cơ sở dữ liệu lưu trữ đa biểu diễn (Multi-representation) bao gồm văn bản thuần để phục vụ tìm kiếm, dữ liệu cấu trúc định dạng JSON để đảm bảo độ trung thực cao khi tái hiện định dạng, mã nguồn HTML để xuất dữ liệu nhanh và khối dữ liệu trạng thái nhị phân của Yjs để khôi phục chính xác lịch sử cộng tác đã chứng minh tính thực tiễn và tối ưu vượt trội. Cách tiếp cận này giúp tối đa hóa hiệu năng hệ thống, giảm thiểu chi phí tính toán lại toàn bộ lịch sử thao tác trên máy chủ, đồng thời cho phép tích hợp hiệu quả cơ chế phân quyền bảo mật chặt chẽ hai lớp từ giao diện người dùng đến tầng dịch vụ WebSocket. Thêm vào đó, việc triển khai thành công cơ chế lưu giữ trạng thái ngoại tuyến tạm thời và tự động hòa trộn dữ liệu qua State Vector khi khôi phục kết nối đã mở ra khả năng thích ứng cao của ứng dụng trong môi trường mạng thiếu ổn định, mang lại trải nghiệm soạn thảo liên tục và an toàn cho người dùng cuối.

Mặc dù hệ thống hiện tại mới chỉ dừng lại ở mô hình thử nghiệm với hạ tầng máy chủ đơn lẻ và cơ sở dữ liệu tập trung, các kết quả đạt được đã đặt nền móng kỹ

thuật cực kỳ vững chắc cho các nghiên cứu tiếp theo. Những hạn chế về khả năng chịu tải cục bộ và sự thiếu vắng các tính năng phân tích thông minh hoàn toàn có thể được khắc phục thông qua định hướng mở rộng hạ tầng phân tán bằng Redis Cluster và tích hợp các mô hình trí tuệ nhân tạo thời gian thực trong tương lai gần. Nhìn chung, đề tài không chỉ giải quyết thành công một bài toán kỹ thuật phức tạp trong môn học Hệ thống phân tán, mà còn chứng minh tính khả thi cao trong việc ứng dụng các giải pháp giải quyết xung đột không cần máy chủ trung tâm điều phối vào các nền tảng ứng dụng cộng tác văn phòng doanh nghiệp thực tế.